

super agent

MetaOrcha SuperAgent — Technical Specification v2.1

Document Status: Implementation-Ready

Supersedes: SuperAgent Technical Specification v1.0, Manifest Evolution & Graph Wiring v1.1, Orchestration Runtime v2.1

Architecture: LangGraph-Powered ReAct Orchestration Framework

Date: March 2026

Changelog v2.1: OpenRouter integration; agent_calls terminology; verbatim tool responses; PnD agent_id resolution contract; three-tier PnD gate; MCP sampling in MCPHandler; auth inside PreFlight; streaming agent events; sequential execution; system tool registry design; file tree; CLI interface; DB schema notes; test harness.

Table of Contents

1. [Executive Summary & Vision](#)
2. [Service Overview Diagrams & File Tree](#)
3. [System Architecture Overview](#)
4. [LangGraph Design & Constructs](#)
5. [Session State Schema](#)
6. [The Orchestrator LLM Node & OpenRouter Integration](#)
7. [PnD Service Interface \(Updated\)](#)
8. [Execution Middleware Pipeline](#)
9. [Agent Handlers \(MCP & A2A — Full Protocol Logic\)](#)
10. [Interrupt System & HITL](#)
11. [Output Normalizer & Data Transformations](#)
12. [System Tool Registry](#)
13. [Task Checklist & Workflow Memory](#)
14. [Workflow Caching & CronJob Templates](#)
15. [Conversational Interface & Context Management](#)
16. [Checklist Mutations & Undo](#)
17. [Conditional Routing & LLM Decision-Making](#)
18. [Artifact Handling](#)
19. [Pre-Flight Checks \(Full Auth Resolution\)](#)
20. [Database Schema Notes](#)
21. [CLI Interface](#)
22. [Test Harness & Test Cases](#)

- 23. [Recommended Models for Production](#)
- 24. [End-to-End Flow Walkthrough](#)

1. Executive Summary & Vision

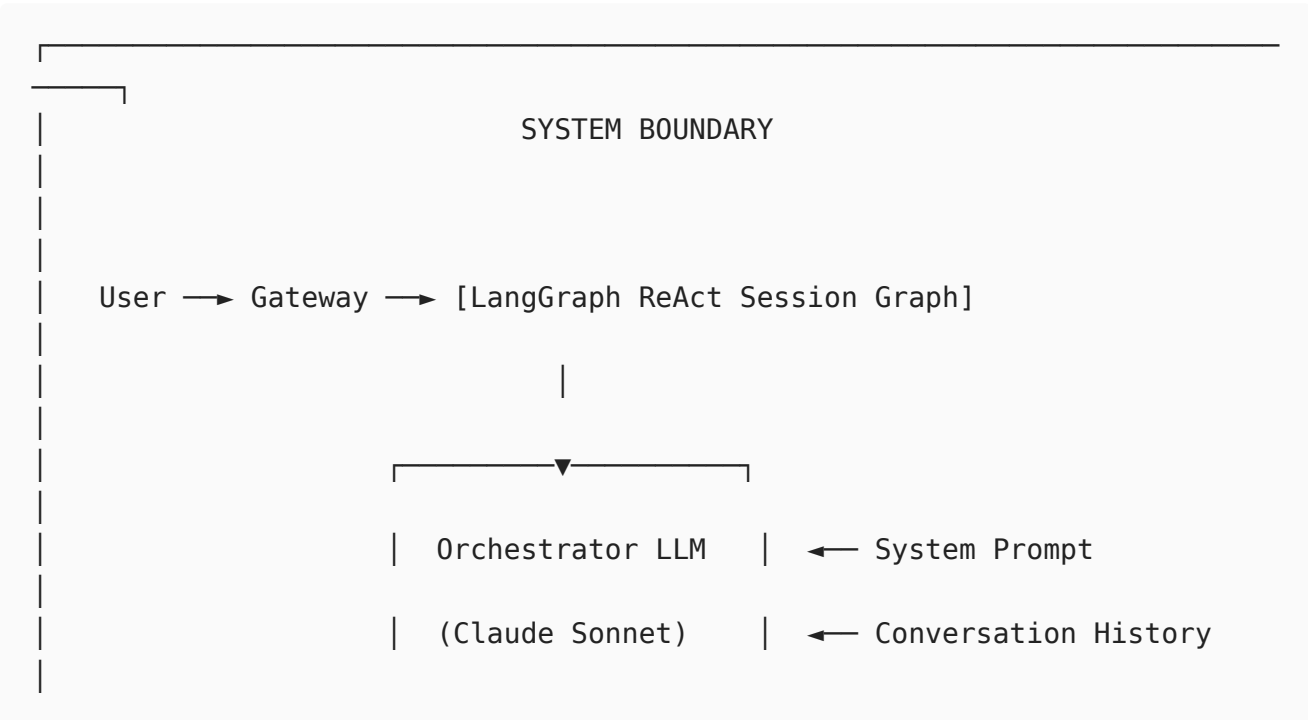
MetaOrcha SuperAgent is a conversational orchestration runtime. Unlike workflow automation platforms (Zapier, n8n, Temporal) that execute pre-defined pipelines, SuperAgent is designed to serve **any workflow it has never seen before**, dynamically composing agent and tool calls in response to natural language, adapting mid-conversation, and handling the full spectrum of complexity — from a single web search to a 20-step multi-agent pipeline spanning three protocols.

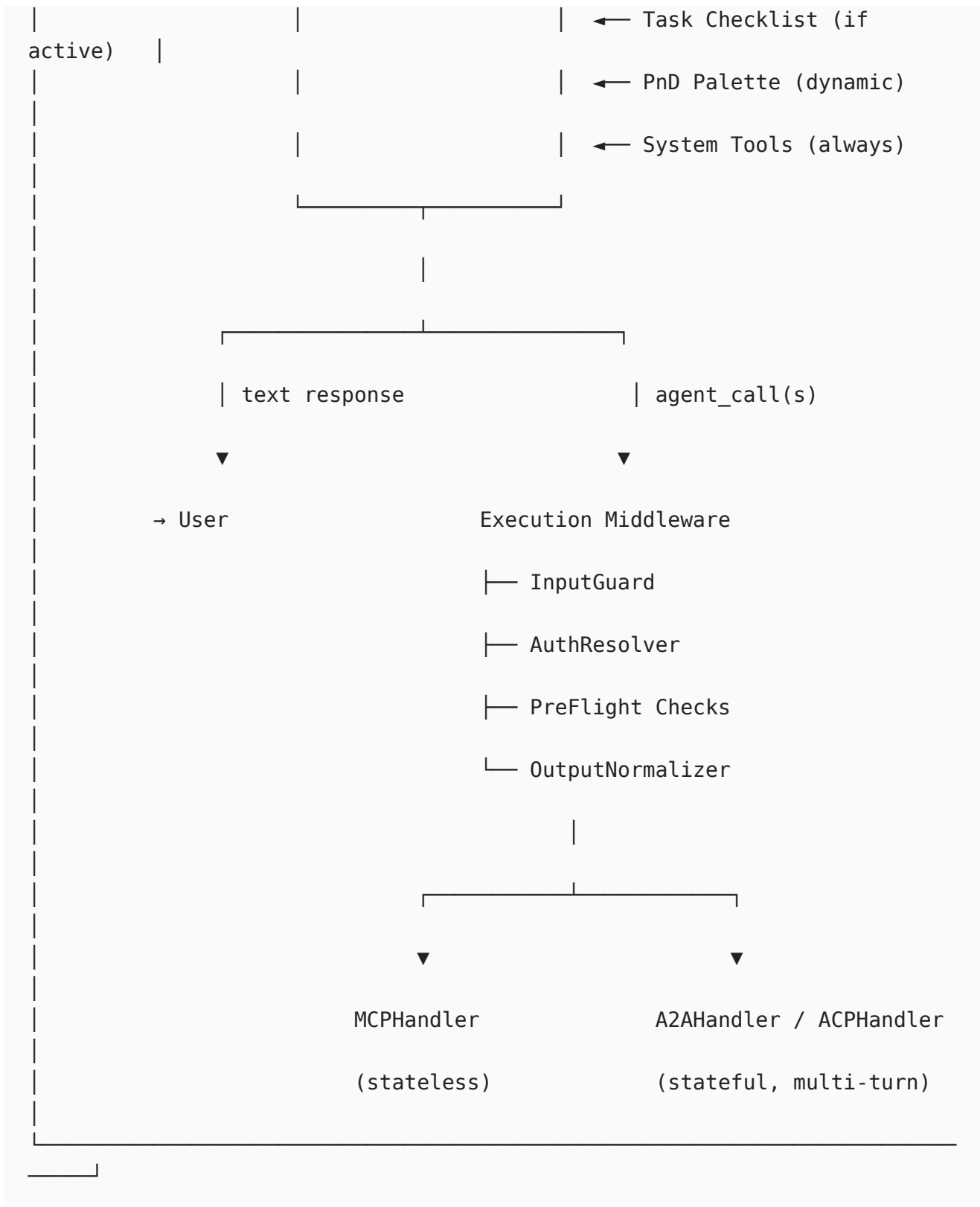
The architecture sits on three foundational principles:

The LLM is the orchestrator. Task decomposition, routing decisions, intermediate reasoning, and error recovery all happen in the LLM's reasoning loop — not in hardcoded system logic. The system's job is to equip the LLM with the right tools, validate every external call, and ensure that failures are surfaced back to the LLM with enough context to self-correct.

LangGraph provides the execution skeleton. LangGraph is not used to encode workflows as DAGs. It is used as a persistent, resumable state machine that manages the ReAct loop, checkpoints session state after every turn, and propagates `NodeInterrupt` events for HITL scenarios. The LLM drives what happens; LangGraph ensures it can always be paused, resumed, and observed.

Execution is governed, not assumed. Every external call — to an MCP tool, A2A agent, or ACP service — passes through a structured middleware pipeline that validates inputs before the call and normalises outputs after it. The LLM never constructs auth headers; they are injected at the last moment by the `PreFlightManager`, invisible to the model. Agent responses that are text or JSON are injected into context verbatim; binary files are registered as artifacts.

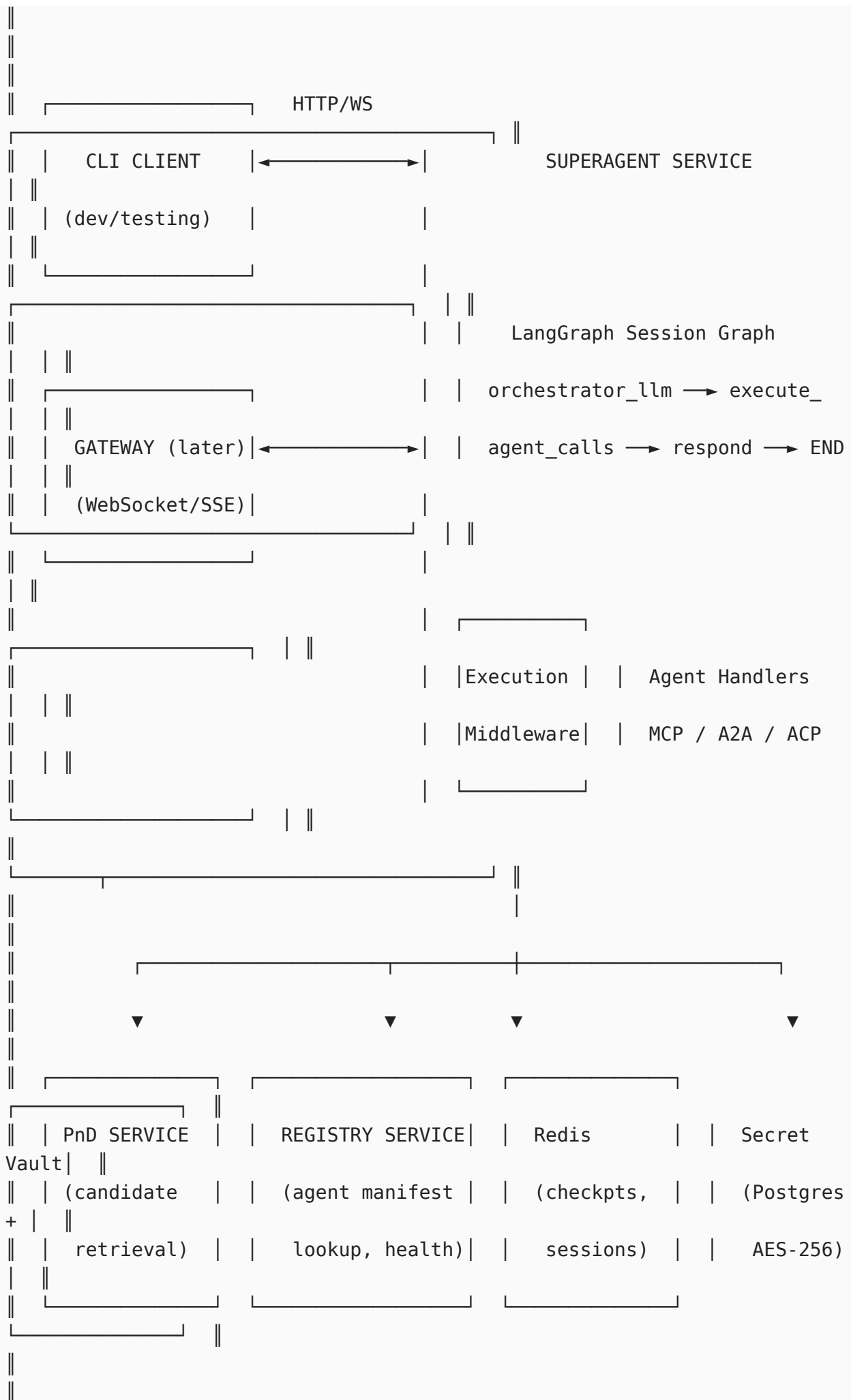




2. Service Overview Diagrams & File Tree

2.1 Full System Context Diagram





```

|| commons/
||
|| └─ db/           (Prisma schema, migrations)
||
|| └─ kafka/       (NOT used by SuperAgent – event bus for other
services) ||
|| └─ types/       (shared Pydantic models, agent_id format, etc.)
||
||
||

```

External agents reached by Agent Handlers:

```

MCP Servers  → SSE endpoint or STDIO process
A2A Agents   → HTTP REST (message/send, tasks/get)
ACP Agents   → (future, same handler interface)

```

OpenRouter API (LLM calls):

```

SuperAgent → https://openrouter.ai/api/v1/chat/completions
            └─ Orchestrator: anthropic/claude-sonnet-4-5
            └─ Small tasks:  anthropic/claude-haiku-4-5

```

2.2 SuperAgent Service File Tree

```

services/
└─ superagent/
    └─ pyproject.toml           # uv-managed, dependencies declared
here
    └─ .env.example           # REDIS_URL, OPENROUTER_API_KEY,
DATABASE_URL, VAULT_KEY
    └─ README.md
    |
    └─ src/
        └─ superagent/
            └─ __init__.py
            └─ main.py         # FastAPI app, startup, dependency
wiring
    |
    | └─ config.py           # Pydantic Settings – reads env vars
    |
    | └─ graph/
    |     └─ __init__.py
    |     └─ builder.py      # build_superagent_graph() – compiled
once at boot
    |
    | └─ state.py           # AgentState TypedDict + all
dataclasses
    |
    | └─ runner.py         # SessionRunner – run_turn(),
resume_from_interrupt()
    |
    |
    | └─ nodes/
    |     └─ __init__.py

```

			└─ orchestrator.py	# orchestrator_llm_node()
			└─ execute_agent_calls.py	# execute_agent_calls_node()
			└─ respond.py	# respond_node()
			└─ middleware/	
			└─ __init__.py	
			└─ pipeline.py	# ExecutionMiddleware – full
pipeline entry point				
			└─ input_guard.py	# InputGuard – schema + type
validation				
			└─ preflight.py	# PreFlightManager – health + auth
(full runtime logic)				
			└─ auth_manager.py	# AuthManager – all 5 auth
strategies				
			└─ output_normalizer.py	# OutputNormalizer – verbatim
text, artifact detection				
			└─ manifest_cache.py	# ManifestCache – 5-min TTL over
Registry calls				
			└─ handlers/	
			└─ __init__.py	
			└─ base.py	# AgentHandler ABC + AgentEvent
types				
			└─ mcp_handler.py	# MCPHandler – sessions, tool
calls, sampling				
			└─ a2a_handler.py	# A2AHandler – task lifecycle,
input-required				
			└─ acp_handler.py	# ACPHandler – stub, mirrors
A2AHandler interface				
			└─ pnd/	
			└─ __init__.py	
			└─ client.py	# PnDClient – HTTP client for PnD
service				
			└─ gate.py	# Three-tier PnD gate (regex →
embedding → Haiku)				
			└─ models.py	# PnDCandidateRequest/Response,
ToolCandidate				
			└─ system_tools/	
			└─ __init__.py	
			└─ registry.py	# SystemToolRegistry – registration
+ dispatch				
			└─ checklist.py	# create_, update_, add_,
abandon_checklist				
			└─ artifacts.py	# read_artifact – PDF, image,
spreadsheet				
			└─ workflow.py	# save_as_workflow_template
			└─ memory.py	# store_to_memory, query_memory

```
(Graphiti)
├── utils.py # web_search, get_datetime,
send_notification
├── context/
│   ├── __init__.py
│   └── window_manager.py # ContextWindowManager -
pagination, compression
├── workflow/
│   ├── __init__.py
│   └── store.py # WorkflowTemplateStore - Postgres
persistence
├── scheduler.py # WorkflowScheduler - cron
execution
├── vault/
│   ├── __init__.py
│   └── client.py # VaultClient - Postgres-backed
AES-256 secrets
├── api/
│   ├── __init__.py
│   └── routes.py # POST /sessions, POST
/sessions/{id}/message, # POST /sessions/{id}/resume, GET
/sessions/{id}/status
├── models.py # Request/Response Pydantic models
└── tests/
    ├── conftest.py # Shared fixtures, mock factories
    ├── unit/
    │   ├── test_input_guard.py
    │   ├── test_output_normalizer.py
    │   ├── test_preflight.py
    │   ├── test_auth_manager.py
    │   ├── test_pnd_gate.py
    │   ├── test_system_tools.py
    │   ├── test_checklist.py
    │   └── test_manifest_cache.py
    ├── integration/
    │   ├── test_mcp_handler.py # Against mock MCP server
    │   ├── test_a2a_handler.py # Against mock A2A server
    │   └── test_execute_node.py # Full middleware + handler
pipeline
├── test_orchestrator_node.py
├── test_session_graph.py # Full graph with MemorySaver
└── e2e/
    └── test_single_turn.py
```

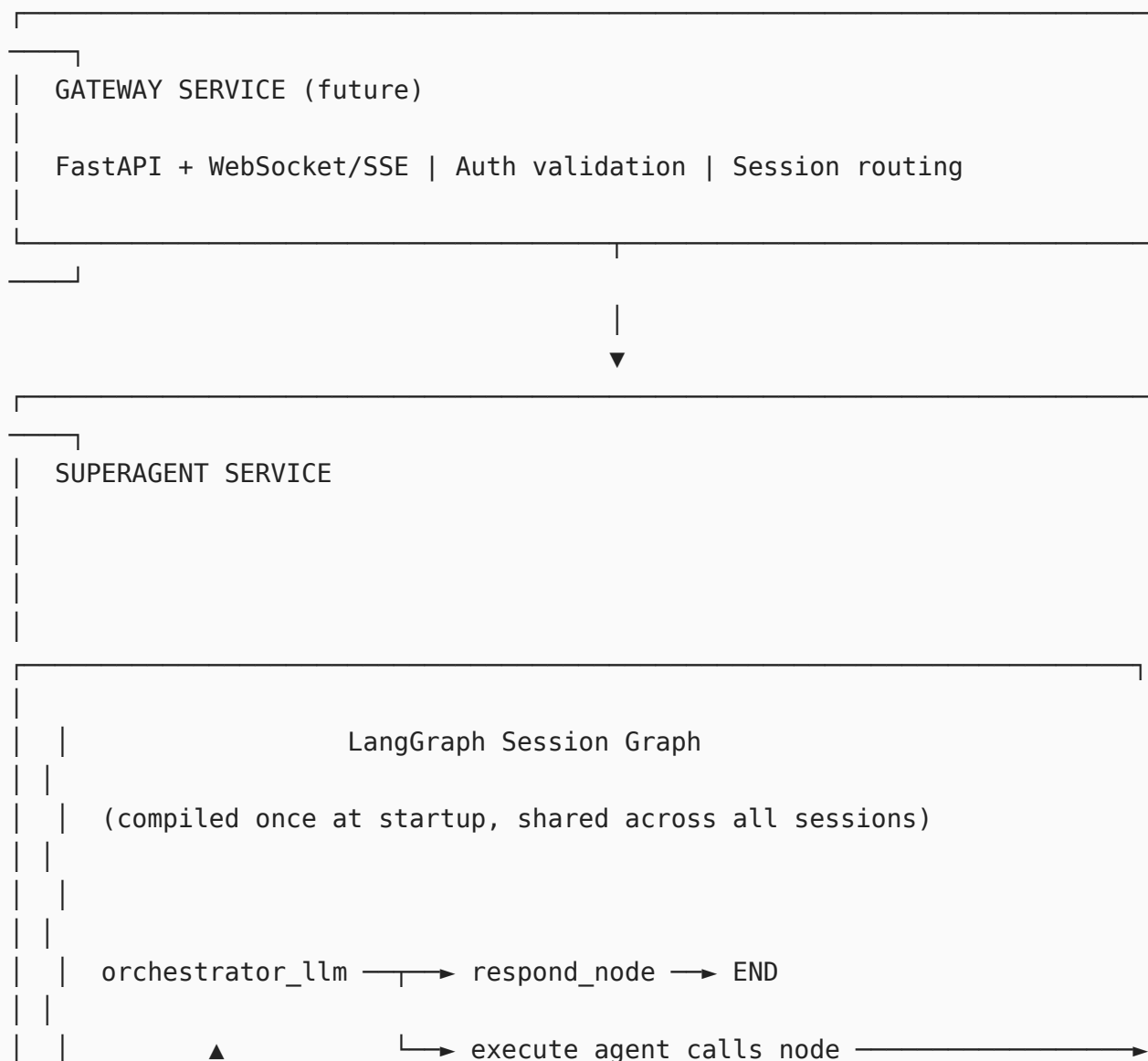
```
├─ test_multi_turn.py
├─ test_hitl_auth.py
├─ test_hitl_approval.py
├─ test_hitl_agent_question.py
├─ test_checklist_mutation.py
└─ test_workflow_template.py
```

2.3 Kafka Usage

SuperAgent does **not** use Kafka. It is a synchronous request-response service accessed via HTTP from the CLI (dev) and later from the Gateway. Events flow back to the caller via HTTP SSE streaming. Kafka is used by other services in the platform (Registry, PnD background jobs) but SuperAgent has no dependency on it.

3. System Architecture Overview

3.1 Component Map



RedisSaver checkpointer – serializes AgentState after every node

EXECUTION MIDDLEWARE

InputGuard

PreFlightManager

(health + auth inside)

OutputNormalizer

AGENT HANDLERS

MCPHandler (stdio/SSE + sampling)

A2AHandler (HTTP + input-required)

ACPHandler (stub, same interface)

CONTEXT WINDOW MANAGER

Pagination enforcement

Light compression fallback

WORKFLOW STORE

Checklist persistence

Workflow template caching

Undo history (snapshots)

PnD SERVICE

SECRET VAULT

Candidate retrieval	
Postgres+AES-256	
(3-stage pipeline)	API keys,
tokens	
Returns agent_ids +	OAuth creds
capability schemas	

3.2 Data Flow Summary

When a user sends a message, the flow is always: Gateway authenticates → session loaded from Redis → LangGraph graph invoked with new message → Orchestrator LLM node runs → if agent calls, `execute_agent_calls_node` fires → results normalized → LLM gets next turn → repeat until done or interrupted → final response streamed to user. Every state transition is checkpointed to Redis so any interruption (auth, HITL, payment) can be resumed without data loss.

4. LangGraph Design & Constructs

3.1 Why LangGraph, and Which Version

We use **LangGraph 0.2.x / 0.3.x** (latest stable). The key constructs we exploit:

StateGraph with typed TypedDict state allows every node to read and write a shared, typed state object. This replaces ad-hoc session dictionaries with a schema-validated state that every component can depend on.

RedisSaver checkpointer serializes the full `AgentState` to Redis after every node completes. This is the foundation of our interrupt/resume capability — if a `NodeInterrupt` is raised anywhere in the graph, LangGraph automatically saves state before propagating the interrupt.

NodeInterrupt is LangGraph's built-in mechanism for pausing graph execution. When raised inside any node, LangGraph stops execution, saves the checkpoint, and returns a `GraphInterrupt` signal to the caller. Resume is done via `graph.invoke(Command(resume=value), config={"thread_id": session_id})`.

Command is LangGraph 0.2+'s way of returning both state updates and routing decisions from a single node, replacing the older `conditional_edges` + separate routing function pattern in some scenarios.

interrupt() (new in LangGraph 0.3) is the functional equivalent of raising `NodeInterrupt`, preferred for new code as it integrates cleanly with the `Command` resume pattern.

MemorySaver vs RedisSaver: `MemorySaver` is for local development and testing only. Production always uses `RedisSaver` for distributed, crash-safe state persistence.

3.2 The Session Graph (Full Definition)

```
# superagent/graph.py
from typing import Annotated, Literal
from typing_extensions import TypedDict
import operator
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.redis import RedisSaver
from langgraph.types import interrupt, Command
from functools import partial

def build_superagent_graph(
    redis_url: str,
    orchestrator_llm,          # via OpenRouter – see Section 23
    small_llm,                 # via OpenRouter – cheaper model for
classification
    pnd_client,
    execution_middleware,
    vault,
    agent_call_result_store,
) -> tuple:
    """
    Builds and compiles the SuperAgent LangGraph session graph.

    Called ONCE at service startup. The compiled graph object is
    stateless and safe to share across all concurrent sessions –
    session-specific state lives entirely in the checkpointer (Redis),
    keyed by thread_id (= session_id).
    """

    graph = StateGraph(AgentState)

    graph.add_node(
        "orchestrator_llm",
        partial(
            orchestrator_llm_node,
            llm=orchestrator_llm,
            small_llm=small_llm,
            pnd_client=pnd_client,
        )
    )

    graph.add_node(
        "execute_agent_calls",
        partial(
            execute_agent_calls_node,
            middleware=execution_middleware,
            vault=vault,
            agent_call_result_store=agent_call_result_store,
```

```

    )
)

graph.add_node("respond", respond_node)

graph.set_entry_point("orchestrator_llm")

graph.add_conditional_edges(
    "orchestrator_llm",
    route_after_orchestrator,
    {
        "agent_calls": "execute_agent_calls",
        "respond": "respond",
    }
)

graph.add_edge("execute_agent_calls", "orchestrator_llm")
graph.add_edge("respond", END)

checkpointer = RedisSaver.from_conn_string(redis_url)
compiled = graph.compile(checkpointer=checkpointer)

return compiled, checkpointer


def route_after_orchestrator(state: "AgentState") ->
Literal["agent_calls", "respond"]:
    """
    Routing function: reads the last AIMessage to determine whether the
    LLM made agent/tool calls or produced a final text response.

    OpenRouter returns tool_calls on the message object regardless of
    which underlying model is used – same field name as Anthropic SDK.
    """
    last_msg = state["messages"][-1]
    if hasattr(last_msg, "tool_calls") and last_msg.tool_calls:
        return "agent_calls"
    return "respond"

```

3.3 How Sessions are Invoked and Resumed

```

# superagent/session_runner.py

class SessionRunner:
    """Manages the lifecycle of a user session against the compiled
    graph."""

    def __init__(self, compiled_graph, redis_client):
        self.graph = compiled_graph

```

```

self.redis = redis_client

async def run_turn(
    self,
    session_id: str,
    user_message: str,
) -> AsyncIterator[StreamEvent]:
    """
    Process one user turn. Streams events back to the gateway.

    The thread_id is the key used by LangGraph's checkpointer to
    identify which session's state to load and save. Always set
    this to the session_id so LangGraph manages continuity
    automatically.
    """
    config = {
        "thread_id": session_id,
        "recursion_limit": 50,  # prevent infinite ReAct loops
    }

    # Prepare the new user message as the input to this turn
    new_input = {
        "messages": [HumanMessage(content=user_message)]
        # All other state fields carry over from the checkpoint
        # automatically
    }

    # Use astream_events (LangGraph 0.2+) to get granular streaming
    # events including token-by-token LLM streaming
    async for event in self.graph.astream_events(
        new_input,
        config=config,
        version="v2",
    ):
        yield self._process_event(event)

async def resume_from_interrupt(
    self,
    session_id: str,
    user_response: str,  # user's answer to the interrupt question
    interrupt_id: str,  # which interrupt we're responding to
) -> AsyncIterator[StreamEvent]:
    """
    Resume a session that was suspended by a NodeInterrupt.

    LangGraph's Command(resume=value) mechanism passes the user's
    response back to the exact point where interrupt() was called,
    then continues execution from there.
    """

```

```

    config = {"thread_id": session_id}

    # Command(resume=value) is the LangGraph 0.3 pattern
    # It resumes the graph from the last interrupt point with the
given value
    resume_input = Command(resume={
        "interrupt_id": interrupt_id,
        "value": user_response,
    })

    async for event in self.graph.astream_events(
        resume_input,
        config=config,
        version="v2",
    ):
        yield self._process_event(event)

```

5. Session State Schema

The `AgentState` is the single source of truth for a session. Every node reads from it and returns partial updates. LangGraph's reducer functions handle merging.

```

# superagent/state.py
from typing import Annotated, TypedDict, Optional, Literal
from typing_extensions import NotRequired
import operator
from langchain_core.messages import BaseMessage
from dataclasses import dataclass, field

# — Reducer functions


---


# LangGraph calls these to merge node return values into the existing
state.

def merge_agent_call_results(existing: dict, new: dict) -> dict:
    """Merge new agent call results without overwriting existing ones."""
    return {**existing, **new}

def append_checklist_history(existing: list, new: list) -> list:
    """Append new undo history entries."""
    return existing + new

# — Core state schema


---



```

```

class AgentState(TypedDict):

    # — Conversation
    

---


    # Standard LangChain messages list. operator.add appends new messages.
    # This is what the Orchestrator LLM receives as its primary context.
    messages: Annotated[list[BaseMessage], operator.add]

    # — Agent Call Result Store
    

---


    # Stores full agent call outputs OUTSIDE the messages list.
    # The messages list receives the full text/JSON response verbatim.
    # Binary artifacts are stored here by artifact_id.
    # Keyed by agent_call_id (== the tool_call_id from OpenRouter
    response).
    # Only used for non-text payloads and artifact references.
    agent_call_result_store: Annotated[dict, merge_agent_call_results]

    # — Task Checklist
    

---


    # Present only when the LLM has started a multi-step task.
    # None for single-turn or pure conversational interactions.
    task_checklist: Optional[TaskChecklist]

    # — Checklist Undo History
    

---


    # Stack of checklist snapshots for undo operations.
    # Each entry is a (timestamp, TaskChecklist) tuple.
    checklist_history: Annotated[list, append_checklist_history]

    # — Interrupt State
    

---


    # Set by execute_agent_calls_node when an interrupt is required.
    # Cleared when the interrupt is resolved via resume.
    pending_interrupt: Optional[PendingInterrupt]

    # — Session Metadata
    

---


    session_id: str
    user_id: str

    # — Context Window Control
    

---


    # Counts total tokens used in the messages list.
    # Used by the Context Window Manager to decide when to compress.
    estimated_token_count: int

    # — Artifact Registry

```

```
# Files produced by agents during this session.
# Keyed by artifact_id. LLM can reference these by ID.
artifacts: dict # artifact_id → ArtifactRef
```

```
# — Workflow Template Capture
```

```
# Populated when the user requests to save the current workflow.
# See Section 12.
captured_workflow: Optional["WorkflowTemplate"]
```

```
# — Supporting data classes
```

```
@dataclass
```

```
class TaskChecklist:
```

```
    checklist_id: str
```

```
    goal: str # "Process inbox for bug reports"
```

```
    steps: list["ChecklistStep"]
```

```
    status: Literal["active", "complete", "abandoned"]
```

```
    created_at: float
```

```
    agents_used: list[str] # agent_ids, populated as steps
```

```
    complete
```

```
    def to_context_string(self) -> str:
```

```
        """Compact string injected into LLM context, not raw JSON."""
```

```
        lines = [f"ACTIVE TASK: {self.goal}", "Steps:"]
```

```
        for step in self.steps:
```

```
            icon = {"pending": "○", "done": "✓", "failed": "✗", "skipped":
```

```
"→"}[step.status]
```

```
            lines.append(f" {icon} [{step.id}] {step.description}")
```

```
            if step.result_summary:
```

```
                lines.append(f"      ↳ {step.result_summary}")
```

```
        return "\n".join(lines)
```

```
@dataclass
```

```
class ChecklistStep:
```

```
    id: int
```

```
    description: str
```

```
    status: Literal["pending", "done", "failed", "skipped"]
```

```
    result_summary: Optional[str] = None # Injected into LLM context
```

```
    result_ref: Optional[str] = None # Redis key for full data
```

```
    agent_call_id: Optional[str] = None # Links to
```

```
agent_call_result_store
```

```
    agent_id: Optional[str] = None # Which agent performed this
```

```
step
```

```
    executed_at: Optional[float] = None
```

```
    is_reversible: bool = False # Can this action be undone?
```

```

undo_context: Optional[dict] = None # Data needed to reverse it

@dataclass
class PendingInterrupt:
    interrupt_id: str
    type: Literal["clarify", "approval", "auth", "agent_question"]
    message: str # Shown to user
    source: str # "orchestrator" | "agent:{agent_id}"
    agent_call_id: str # The pending agent call this
interrupt blocks
    metadata: dict = field(default_factory=dict)
    # For "approval": contains action_description, payload_preview
    # For "auth": contains oauth_url, tool_id
    # For "agent_question": contains original agent question, options

@dataclass
class ArtifactRef:
    artifact_id: str
    artifact_type: Literal["pdf", "image", "audio", "spreadsheet",
"document", "url", "text", "json"]
    session_id: str
    url: Optional[str] = None
    inline_b64: Optional[str] = None # For small artifacts only
    mime_type: Optional[str] = None
    filename: Optional[str] = None
    size_bytes: Optional[int] = None
    metadata: dict = field(default_factory=dict)
    created_at: float = field(default_factory=lambda: time.time())

```

6. The Orchestrator LLM Node & OpenRouter Integration

This is the brain of the system. It receives the full conversation context, the active task checklist, the dynamically-injected tool palette, and the system prompt — and decides what to do next.

6.1 OpenRouter Integration

All LLM calls in SuperAgent go through **OpenRouter** (<https://openrouter.ai/api/v1>). OpenRouter exposes a unified OpenAI-compatible API, so we use the `openai` Python SDK pointed at OpenRouter's base URL. Model references are configuration values — never hardcoded strings.

```

# superagent/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # OpenRouter
    openrouter_api_key: str

```

```

openrouter_base_url: str = "https://openrouter.ai/api/v1"
orchestrator_model: str = "anthropic/claude-sonnet-4-5"
small_model: str = "anthropic/claude-haiku-4-5"

# Redis
redis_url: str

# Database (shared Postgres via commons/db)
database_url: str

# Vault encryption key
vault_key: str

# PnD service
pnd_service_url: str

class Config:
    env_file = ".env"

settings = Settings()

```

```

# superagent/nodes/orchestrator.py
from openai import AsyncOpenAI

def make_llm_client(settings: Settings) -> AsyncOpenAI:
    """
    Returns an OpenAI-compatible client pointed at OpenRouter.
    OpenRouter accepts the same request format as the OpenAI API,
    including the `tools` parameter for function calling.
    """
    return AsyncOpenAI(
        api_key=settings.openrouter_api_key,
        base_url=settings.openrouter_base_url,
    )

```

Tool schema format: OpenRouter uses the OpenAI function-calling format. All tool schemas throughout the system — from PnD candidates to system tools — must conform to this structure:

```

# OpenRouter / OpenAI tool schema format (NOT Anthropic format)
{
    "type": "function",
    "function": {
        "name": "gmail-mcp-01__fetch_emails",
        "description": "[Gmail MCP] Fetch emails from inbox with optional filters.",
        "parameters": {
            # ← "parameters" not "input_schema"
            "type": "object",
            "properties": {

```

```

        "query": {"type": "string", "description": "Gmail search
query"},
        "max_results": {"type": "integer", "default": 50},
    },
    "required": ["query"]
}
}
}

```

6.2 Context Assembly

```

async def orchestrator_llm_node(
    state: AgentState,
    llm: AsyncOpenAI,
    small_llm: AsyncOpenAI,
    pnd_client,
) -> dict:
    """
    The core reasoning node. Called on every turn and after every
    agent call result. Decides: respond, call an agent, ask for
    clarification, or update the task checklist.
    """

    # Step 1: Three-tier PnD gate (see 6.3)
    needs_external = await _pnd_gate(state, small_llm)

    # Step 2: Build tool palette (OpenAI format throughout)
    system_tools = SYSTEM_TOOL_REGISTRY.get_all_schemas() # OpenAI
    format

    pnd_tools = []
    if needs_external:
        pnd_response = await pnd_client.get_candidates(
            query=_extract_latest_intent(state["messages"]),
            conversation_context=_get_recent_context(state["messages"],
n=3),
            user_id=state["user_id"],
            top_k=8,
        )
        pnd_tools = pnd_response.to_openai_tool_schemas() # See Section
7

    all_tools = system_tools + pnd_tools

    # Step 3: Build message list with context management
    messages = _build_llm_messages(state)

    # Step 4: Call LLM via OpenRouter
    response = await llm.chat.completions.create(

```

```

        model=settings.orchestrator_model,
        messages=messages,
        tools=all_tools if all_tools else None,
        tool_choice="auto",
        max_tokens=4096,
        temperature=0.3,
        extra_headers={
            "HTTP-Referer": "https://emergeplatform.io",
            "X-Title": "MetaOrcha SuperAgent",
        }
    )

    choice = response.choices[0]
    ai_message = _openrouter_response_to_langchain(choice)

    new_token_count = (
        state["estimated_token_count"]
        + response.usage.total_tokens
    )

    return {
        "messages": [ai_message],
        "estimated_token_count": new_token_count,
    }

def _build_llm_messages(state: AgentState) -> list[dict]:
    """
    Converts AgentState messages to OpenRouter message format.
    Injects system prompt and active task checklist.
    Applies context window management (pagination hints for large
    results).
    """
    system_content = _build_system_prompt(state)
    if state.get("task_checklist"):
        system_content += (
            "\n\n[TASK CONTEXT]\n"
            + state["task_checklist"].to_context_string()
        )

    result = [{"role": "system", "content": system_content}]

    # Convert LangChain messages → OpenRouter dicts
    managed = ContextWindowManager.compress_if_needed(
        messages=state["messages"],
        token_budget=140_000,
    )
    for msg in managed:
        result.append(_langchain_to_openrouter(msg))

```

```
return result
```

6.3 Three-Tier PnD Gate

The gate decides whether PnD should be called for this turn. It costs nothing for obvious cases.

```
# superagent/pnd/gate.py
import re
from sentence_transformers import SentenceTransformer, util as st_util

_EMBED_MODEL = SentenceTransformer("all-MiniLM-L6-v2") # local, ~80MB,
~5ms

# Phrases that definitively DON'T need external agents
_NO_TOOL_RE = re.compile(
    r"\b(explain|what is|what are|how does|thanks|thank you|ok|got it"
    r"|what did you|what step|status|continue|proceed|sure|yes|no)\b",
    re.IGNORECASE,
)

# Phrases that definitively DO need external agents
_TOOL_RE = re.compile(
    r"\b(search|fetch|create|send|delete|update|read my|check my|get my"
    r"|download|upload|book|schedule|analyze|generate|find|run|execute)\b",
    re.IGNORECASE,
)

# Embeddings of "tool-requiring" anchor phrases (computed once at import)
_TOOL_ANCHORS = _EMBED_MODEL.encode([
    "I need to call an external service to do this",
    "This requires fetching data from an API",
    "I should use a tool for this action",
], convert_to_tensor=True)

async def _pnd_gate(state: AgentState, small_llm: AsyncOpenAI) -> bool:
    """
    Three-tier gate deciding whether to call PnD this turn.

    Tier 1 – Rule-based (0ms, 0 cost):
    - Active checklist always fetches (task is in progress)
    - Explicit NO patterns → skip
    - Explicit TOOL patterns → fetch

    Tier 2 – Semantic similarity (5ms, 0 cost):
    - Embed latest message, compare to tool-requiring anchors
    - Score > 0.6 → fetch; Score < 0.3 → skip
```

```

Tier 3 – Haiku classification (100–200ms, ~$0.00001):
- Only for ambiguous range (0.3–0.6)
"""
latest = state["messages"][-1].content if state["messages"] else ""

# Tier 1a: active checklist → always look for relevant agents
if state.get("task_checklist") and state["task_checklist"].status ==
"active":
    return True

# Tier 1b: definitive NO patterns
if _NO_TOOL_RE.search(latest) and not _TOOL_RE.search(latest):
    return False

# Tier 1c: definitive TOOL patterns
if _TOOL_RE.search(latest):
    return True

# Tier 2: local embedding similarity
query_emb = _EMBED_MODEL.encode(latest[:500], convert_to_tensor=True)
scores = st_util.cos_sim(query_emb, _TOOL_ANCHORS)[0]
max_score = float(scores.max())

if max_score > 0.6:
    return True
if max_score < 0.3:
    return False

# Tier 3: Haiku for ambiguous range
resp = await small_llm.chat.completions.create(
    model=settings.small_model,
    messages=[
        {"role": "system", "content": "Answer only YES or NO."},
        {"role": "user", "content":
            f"Does this request require calling an external tool, API,
            f"or agent to fulfil? Request: '{latest[:300]}'"}
    ],
    max_tokens=5,
    temperature=0.0,
)
return "YES" in resp.choices[0].message.content.upper()

```

6.4 The Orchestrator System Prompt

```

SYSTEM_PROMPT_TEMPLATE = """
You are MetaOrcha, an intelligent orchestration assistant for EmergeOS.
You help users accomplish tasks by reasoning over their requests and

```

delegating
to specialized agents and tools when needed.

CORE DIRECTIVES

1. Clarify Before Acting on Ambiguous Multi-Step Tasks

Before starting any task involving multiple agent calls or external services,
confirm your interpretation with ONE clear, specific question if anything is
ambiguous. Do NOT ask multiple questions at once.

2. Use Your Own Reasoning for Intermediate Steps

Not every step requires an external agent. Classification, summarization,
decision-making, and formatting can all be done by you directly.

3. ONE AGENT CALL PER TURN – STRICT SEQUENTIAL EXECUTION

Make only ONE agent call per turn. After receiving the result, decide what
to
do next. NEVER call an agent whose inputs depend on the output of another
agent call you have not yet completed. This prevents hallucinated inputs.

4. Maintain a Task Checklist for Multi-Step Work

When starting a task with 3 or more steps, use `system::create_checklist`.
Update it as you complete each step.

5. Never Fabricate Agent Inputs

Only call agents with inputs you can derive from the conversation or from
a previous agent call result you have already received.

6. Confirm Before Irreversible Actions

Any irreversible action must be confirmed by the user. State clearly what
you are about to do and ask "Shall I proceed?"

7. Surface Agent Questions Faithfully

If an external agent sends a question (ToolMessage prefixed "[AGENT
QUESTION]"),
pass it to the user verbatim. Do NOT answer on the user's behalf.

8. Scope Responses Appropriately

For general knowledge questions requiring no agents, answer directly.
""

7. PnD Service Interface (Updated)

7.1 What Changes and Why

The PnD service's **internal 3-stage retrieval pipeline is unchanged**: GIN filter → HNSW vector search → cross-encoder reranking. What changes is the output contract. PnD returns a **palette of candidate agents and their capabilities** alongside their `agent_id`s. The SuperAgent middleware uses these `agent_id`s to fetch full manifests (transport, auth) from the Registry at call time.

7.2 agent_id as the Resolution Key

The Prisma schema shows agents are keyed by `id` in DID format (`did:emerge:agent:xyz`). Because colons are not valid in OpenRouter tool names, the tool name uses a sanitised form: colons and slashes stripped, lowercase. The full DID is carried on the `ToolCandidate` for middleware resolution.

```
DID:      did:emerge:agent:gmail-mcp-01
Tool name: did-emerge-agent-gmail-mcp-01__fetch_emails
Parsing:  split on "__", left part = sanitised agent_id → look up DID in
ToolCandidate
```

7.3 Request Schema

```
# superagent/pnd/models.py
@dataclass
class PnDCandidateRequest:
    query: str
    conversation_context: list[str] # Last 3 turns as strings
    user_id: str
    top_k: int = 8
    protocol_filter: Optional[str] = None # "MCP" | "A2A" | None
    exclude_agent_ids: list[str] = field(default_factory=list)
```

7.4 Response Schema

```
@dataclass
class PnDCandidateResponse:
    candidates: list["ToolCandidate"]
    retrieval_latency_ms: int

    def to_openai_tool_schemas(self) -> list[dict]:
        """
        Converts candidates into OpenRouter / OpenAI function-calling
        format.

        MCP agents: each Capability of type TOOL becomes a separate
        callable function.
        A2A agents: the whole agent becomes a single
        "delegate_{sanitised_id}" function.
        """
```

```

schemas = []
for c in self.candidates:
    sanitised = _sanitise_agent_id(c.agent_id)
    if c.protocol_type == "MCP":
        for cap in c.capabilities:
            if cap.capability_type != "TOOL":
                continue
            schemas.append({
                "type": "function",
                "function": {
                    "name": f"{sanitised}__{cap.capability_id}",
                    "description": f"[{c.agent_name}]
{cap.description}",
                    "parameters": cap.input_schema or {"type":
"object", "properties": {}},
                }
            })
        else: # A2A / ACP
            cap_summary = ", ".join(cap.name for cap in
c.capabilities[:5])
            schemas.append({
                "type": "function",
                "function": {
                    "name": f"delegate__{sanitised}",
                    "description": (
                        f"[{c.protocol_type} Agent] {c.agent_name}:
{c.agent_description}. "
                        f"Capabilities: {cap_summary}. "
                        f"Provide a natural language task
description."
                    ),
                    "parameters": {
                        "type": "object",
                        "properties": {
                            "task": {
                                "type": "string",
                                "description": "Natural language
description of what to do"
                            }
                        },
                    },
                    "required": ["task"]
                }
            })
    return schemas

```

```

def _sanitise_agent_id(agent_id: str) -> str:
    """did:emerge:agent:gmail-mcp-01 → did-emerge-agent-gmail-mcp-01"""

```

```

    return agent_id.replace(":", "-").replace("/", "-").lower()

@dataclass
class ToolCandidate:
    agent_id: str  # Full DID – used by middleware for
    manifest lookup
    agent_name: str
    agent_description: str
    protocol_type: str  # "MCP" | "A2A"
    relevance_score: float
    capabilities: list["CandidateCapability"]  # From Capability table
    health_status: str  # From Agent.health_status – PnD only
    returns HEALTHY

@dataclass
class CandidateCapability:
    capability_id: str  # Matches Capability.capability_id in
    DB
    capability_type: str  # "TOOL" | "RESOURCE" | "PROMPT"
    name: str
    description: str
    input_schema: Optional[dict]  # For TOOL type
    output_schema: Optional[dict]  # For TOOL type

```

7.5 Manifest Resolution in Middleware

When the middleware receives an agent call, it resolves the full manifest from the Registry using the `agent_id`. This uses a local `ManifestCache` (5-minute TTL, matching the runtime doc):

```

# superagent/middleware/manifest_cache.py

class ManifestCache:
    """
    Local cache of agent manifests in the SuperAgent service.
    Reduces calls to the Registry for repeated agent invocations.
    TTL = 5 minutes (matching the runtime spec).
    """
    TTL = 300  # seconds

    def __init__(self, redis: Redis, registry_client):
        self.redis = redis
        self.registry = registry_client

    async def get_manifest(self, agent_id: str) -> dict:
        """
        Returns the full agent manifest including transport, security,
        capabilities, and payment config.

```

```

    Raises ValueError if agent not found.
    """
    cache_key = f"manifest:{agent_id}"
    cached = await self.redis.get(cache_key)
    if cached:
        data = json.loads(cached)
        if time.time() - data["cached_at"] < self.TTL:
            return data["manifest"]

    # Fetch from Registry service (HTTP)
    manifest = await self.registry.get_agent_manifest(agent_id)
    if not manifest:
        raise ValueError(f"Agent {agent_id} not found in registry")

    await self.redis.set(
        cache_key,
        json.dumps({"manifest": manifest, "cached_at": time.time()}),
        ex=self.TTL,
    )
    return manifest


def parse_agent_call(tool_name: str, candidates: list[ToolCandidate]) ->
tuple[str, str, str]:
    """
    Parses a tool name into (agent_id, capability_id, protocol_type).

    Examples:
    "did-emerge-agent-gmail-mcp-01_fetch_emails"
    → ("did:emerge:agent:gmail-mcp-01", "fetch_emails", "MCP")
    "delegate__did-emerge-agent-jira-a2a-01"
    → ("did:emerge:agent:jira-a2a-01", None, "A2A")
    """
    if tool_name.startswith("system::"):
        return (None, tool_name, "SYSTEM")

    is_delegation = tool_name.startswith("delegate__")
    if is_delegation:
        sanitised = tool_name.removeprefix("delegate__")
        capability_id = None
    else:
        parts = tool_name.split("__", 1)
        sanitised, capability_id = parts[0], parts[1]

    # Find the original agent_id from candidates
    candidate = next(
        (c for c in candidates if _sanitise_agent_id(c.agent_id) ==
sanitised),
        None

```

```

    )
    if not candidate:
        raise ValueError(f"Agent not found for tool name: {tool_name}")

    return (candidate.agent_id, capability_id, candidate.protocol_type)

```

@dataclass

class ToolCandidate:

protocol_type: Literal["mcp", "a2a", "acp"]

relevance_score: float

```

# For MCP servers
server_id: Optional[str] = None
server_name: Optional[str] = None
tools: list["MCPToolSchema"] = field(default_factory=list)

# For A2A / ACP agents
agent_id: Optional[str] = None
agent_name: Optional[str] = None
agent_description: Optional[str] = None
capability_summary: list[str] = field(default_factory=list)
transport_endpoint: Optional[str] = None # Resolved by PnD, opaque to LLM

```

@dataclass

class MCPToolSchema:

name: str

description: str

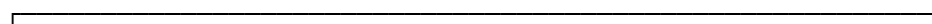
input_schema: dict # JSON Schema object

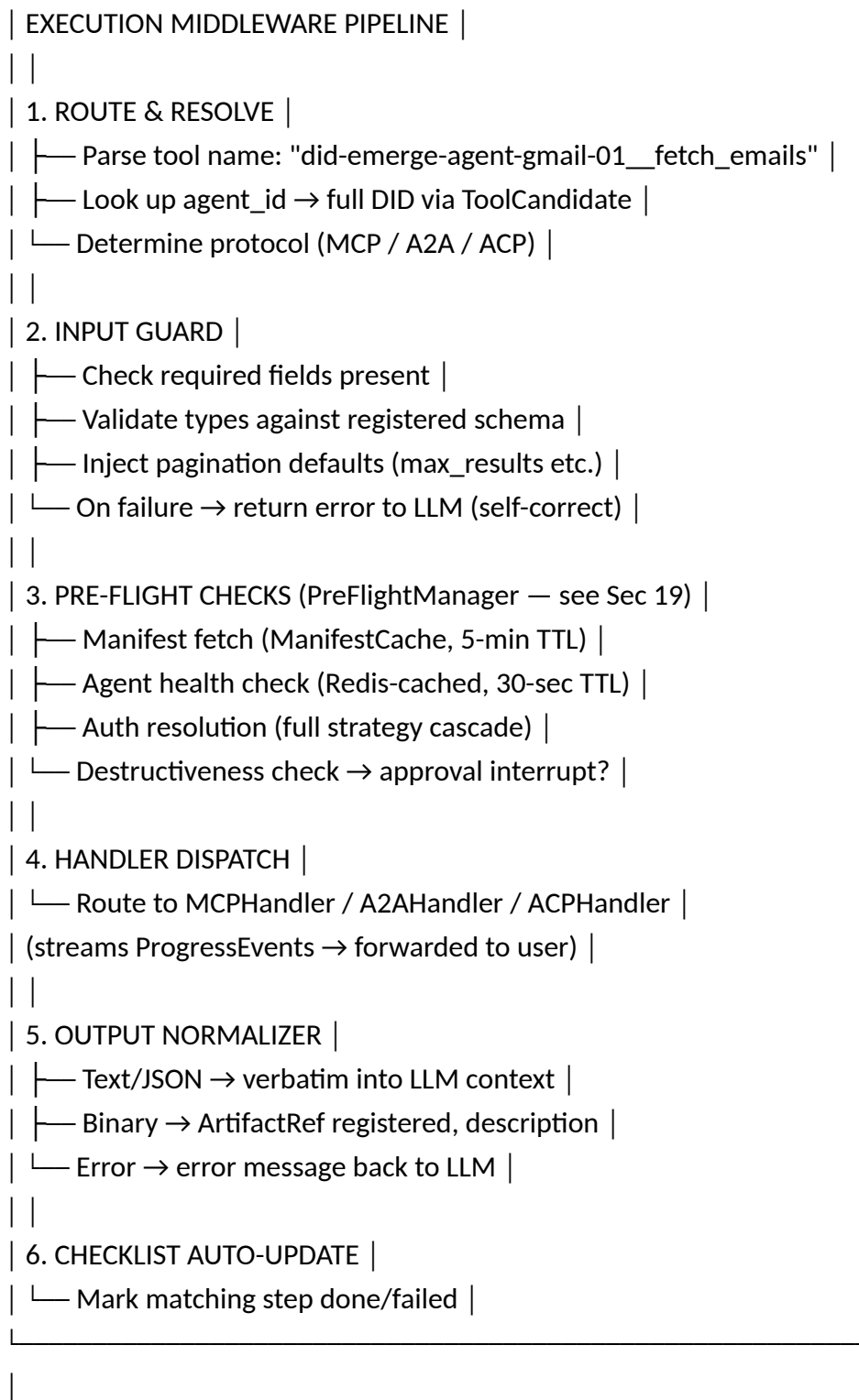
is_destructive: bool = False # Flags irreversible tools for ApprovalCheck

8. Execution Middleware Pipeline

Every agent call emitted by the Orchestrator LLM flows through this pipeline synchronously before any external system is contacted. Think of it as a smart gate that validates, enriches, and governs every outbound call.

LLM agent_call





▼
agent_call result injected into LLM context (verbatim text or artifact ref)

```
```python
superagent/middleware.py

class ExecutionMiddleware:

 def __init__(self, registry, vault, agent_call_result_store,
health_cache):
 self.registry = registry
```

```

self.vault = vault
self.store = agent_call_result_store
self.health_cache = health_cache

async def execute(
 self,
 tool_call: ToolCall,
 state: AgentState,
) -> AgentCallResult:
 """
 Full middleware pipeline for a single tool call.
 Returns an AgentCallResult that will be injected into the LLM's
context.
 """

 # — Step 1: Route & Resolve

 resolved = self.registry.resolve(agent_call.name)
 # resolved: {protocol, server_id/agent_id, tool_name, schema,
is_destructive}

 # — Step 2: Input Guard

 guard_result = InputGuard.validate(
 inputs=agent_call.input,
 schema=resolved.input_schema,
)
 if not guard_result.passed:
 # Return error as tool_result – LLM self-corrects
 return AgentCallResult(
 tool_call_id=agent_call.id,
 success=False,
 summary=guard_result.llm_feedback, # Clear error message
 structured_data=None,
 error_type="input_validation",
)

 # — Step 3: Pre-Flight Checks

 preflight = await PreFlightChecker.check(
 resolved=resolved,
 user_id=state["user_id"],
 health_cache=self.health_cache,
)

 if preflight.requires_auth:
 # Suspend and ask user to authorize
 # This raises NodeInterrupt internally via interrupt()
 await _handle_auth_interrupt(preflight, tool_call, state)

```

```

 # Execution pauses here – resumes when auth completes

 if preflight.requires_approval and resolved.is_destructive:
 await _handle_approval_interrupt(resolved, tool_call, state)
 # Execution pauses here – resumes when user approves

 if not preflight.agent_healthy:
 return AgentCallResult(
 tool_call_id=agent_call.id,
 success=False,
 summary=f"Agent {resolved.agent_id} is currently
unavailable. "
 f"Health check failed: {preflight.health_error}",
 error_type="agent_unavailable",
)

 # — Step 4: Auth Resolution

 auth_context = await self.vault.resolve_auth(
 agent_id=resolved.agent_id or resolved.server_id,
 user_id=state["user_id"],
)
 # auth_context contains credentials – NEVER logged, NEVER in LLM
context

 # — Step 5: Handler Dispatch

 handler = HandlerFactory.get(resolved.protocol)

 # Handlers yield AgentEvents (see Section 8)
 raw_output = None
 async for event in handler.invoke(
 task_or_inputs=agent_call.input,
 resolved=resolved,
 auth_context=auth_context,
 state=state,
):
 if isinstance(event, ProgressEvent):
 await _stream_progress_to_user(event, state["session_id"])

 elif isinstance(event, ClarifyEvent):
 # Agent is asking a question – surface to user as
interrupt
 await _handle_agent_clarify_interrupt(event, tool_call,
state)

 # Graph suspends here

 elif isinstance(event, ArtifactEvent):
 # Register artifact and return ref, don't inline binary

```

```

data
 artifact_id = _register_artifact(event.artifact_ref,
state)
 # Store in state for later reference by LLM

 elif isinstance(event, FinalResultEvent):
 raw_output = event.output

— Step 6: Output Normalizer

 normalized = await OutputNormalizer.normalize(
 raw_output=raw_output,
 schema=resolved.output_schema,
 tool_name=agent_call.name,
)

 # Store full data in Redis (by reference, not inline)
 result_key = f"result:{state['session_id']}:{agent_call.id}"
 await self.store.set(result_key, normalized.structured_data)

— Step 7: Checklist Update

 updated_checklist = _update_checklist_step(
 checklist=state.get("task_checklist"),
 tool_call_id=agent_call.id,
 result_summary=normalized.summary,
 result_ref=result_key,
 agent_id=resolved.agent_id or resolved.server_id,
 success=normalized.success,
)

 return AgentCallResult(
 tool_call_id=agent_call.id,
 success=normalized.success,
 summary=normalized.summary, # Goes into LLM context
 structured_data_ref=result_key, # Redis key – NOT in LLM
context
 artifacts=normalized.artifacts,
 updated_checklist=updated_checklist,
)

```

## 9. Agent Handlers (MCP & A2A — Full Protocol Logic)

All handlers implement the universal `AgentHandler` interface. The Execution Middleware talks exclusively through this interface, making protocol differences invisible to the rest of the system.

### 9.1 Universal Interface

```

superagent/handlers/base.py
from abc import ABC, abstractmethod
from typing import AsyncIterator
from dataclasses import dataclass, field

class AgentHandler(ABC):

 @abstractmethod
 async def invoke(
 self,
 task_or_inputs: dict | str,
 resolved: "ResolvedAgent",
 auth_context: "AuthContext",
 state: "AgentState",
) -> AsyncIterator["AgentEvent"]:
 """Primary invocation. Yields events until complete or
interrupted."""
 ...

 @abstractmethod
 async def resume(
 self,
 resume_context: dict,
 user_response: str,
 auth_context: "AuthContext",
) -> AsyncIterator["AgentEvent"]:
 """Resume after a ClarifyEvent interrupt was resolved."""
 ...

 @abstractmethod
 async def cancel(self, agent_session_id: str) -> None:
 """Cancel an in-progress agent interaction."""
 ...

Universal event types
@dataclass
class ProgressEvent:
 message: str # "Searching your inbox..."

@dataclass
class ClarifyEvent:
 question: str # From the external agent
 options: list[str] # Optional multiple choice
 resume_context: dict # Protocol-specific context to resume
with

@dataclass
class ArtifactEvent:

```

```

 artifact_ref: "ArtifactRef"

@dataclass
class FinalResultEvent:
 output: dict # The actual result data
 success: bool = True

@dataclass
class ErrorEvent:
 message: str
 retryable: bool = False
 error_code: Optional[str] = None

```

## 9.2 MCP Handler

MCP is stateless: each tool call is an independent RPC. The handler manages connection pooling (not per-call connection setup) and handles the `initialize` → `call_tool` → `result` lifecycle.

```

superagent/handlers/mcp_handler.py
from mcp import ClientSession, StdioServerParameters
from mcp.client.sse import sse_client
from mcp.client.stdio import stdio_client

Sampling classifier prompt (from runtime doc)
SAMPLING_CLASSIFIER_PROMPT = """
You are an orchestration router. An AI agent has sent the following
request:
"{agent_prompt}"

Analyse if this requires EXPLICIT HUMAN APPROVAL (e.g., spending money,
deleting data, security decisions, moral/ethical choices, ambiguous
requests)
or if it is a simple LOGIC/KNOWLEDGE task (e.g., summarisation,
calculation,
formatting, research).

Respond ONLY with valid JSON:
{"needs_human": boolean, "reason": string}

Examples:
- "Should I delete 'production_db.sql'?" → {"needs_human": true,
"reason": "Destructive action"}
- "Summarise this 500-page report" → {"needs_human": false, "reason":
"Routine task"}
"""

class MCPHandler(AgentHandler):
 """

```

Handles MCP (Model Context Protocol) agent calls.

MCP is session-based: each server requires an initialize → initialized handshake before tools/call can be invoked. Sessions are pooled.

Supports sampling/createMessage: MCP agents can request LLM assistance mid-execution. The sampling classifier decides whether to auto-respond (using the system small LLM) or surface the request to the user as a HITL interrupt.

Transport: SSE (HTTP) or STDIO (local process)  
"""

```
def __init__(self, connection_pool: "MCPConnectionPool", small_llm:
AsyncOpenAI):
 self.pool = connection_pool
 self.small_llm = small_llm
 self._pending_sampling_interrupt: Optional[dict] = None

 async def invoke(
 self,
 task_or_inputs: dict,
 resolved: "ResolvedAgent",
 auth_context: "AuthContext",
 state: "AgentState",
) -> AsyncIterator[AgentEvent]:

 transport = resolved.manifest["transport"]
 capability_id = resolved.capability_id

 yield ProgressEvent(message=f"Connecting to
{resolved.agent_name}...")

 try:
 if transport["type"] == "SSE":
 async with sse_client(
 transport["endpoint"],
 headers=auth_context.headers,
) as (read, write):
 async with ClientSession(read, write) as session:
 async for event in self._execute_session(
 session, task_or_inputs, capability_id, state,
 resume_context=None,
):
 yield event

 elif transport["type"] == "STDIO":
 server_params = StdioServerParameters(
 command=transport["command"],
```

```

 args=transport.get("args", []),
)
 async with stdio_client(server_params) as (read, write):
 async with ClientSession(read, write) as session:
 async for event in self._execute_session(
 session, task_or_inputs, capability_id, state,
 resume_context=None,
):
 yield event
 else:
 yield ErrorEvent(
 message=f"Unsupported MCP transport:
{transport['type']}",
 retryable=False,
)

 except Exception as e:
 yield ErrorEvent(message=f"MCP handler error: {str(e)}",
retryable=True)

 async def _execute_session(
 self,
 session: ClientSession,
 inputs: dict,
 capability_id: str,
 state: "AgentState",
 resume_context: Optional[dict],
) -> AsyncIterator[AgentEvent]:
 """Execute within an established MCP session with sampling
support."""

 # 1. Initialize session
 await session.initialize()
 yield ProgressEvent(message="Session initialised. Invoking
tool...")

 # 2. Register sampling callback BEFORE invoking the tool
 # The MCP SDK fires this callback if the agent calls
sampling/createMessage
 sampling_result_holder = {"result": None, "interrupt_needed":
False}

 async def _handle_sampling_request(request_id: str, messages:
list):
 """
 Intercepts sampling/createMessage from the MCP agent.
 Classifies whether human approval is needed.
 """
 agent_prompt = messages[0]["content"]["text"] if messages else

```

```
"""
```

```
 # Run cheap classifier
 resp = await self.small_llm.chat.completions.create(
 model=settings.small_model,
 messages=[
 {"role": "user", "content":
SAMPLING_CLASSIFIER_PROMPT.format(
 agent_prompt=agent_prompt[:500]
)}
],
 max_tokens=60,
 temperature=0.0,
)

 import json as _json
 try:
 classification =
_json.loads(resp.choices[0].message.content)
 except Exception:
 classification = {"needs_human": True, "reason": "Parse
error"}

 if classification["needs_human"]:
 # Signal to the outer loop that we need a HITL interrupt
 sampling_result_holder["interrupt_needed"] = True
 sampling_result_holder["interrupt_payload"] = {
 "type": "mcp_sampling",
 "request_id": request_id,
 "agent_prompt": agent_prompt,
 "reason": classification["reason"],
 }
 # We cannot await an interrupt here (we're in a sync SDK
callback)

 # We return a placeholder; the outer loop will detect the
flag and suspend
 return {"role": "assistant", "content": {"type": "text",
"text": "__PENDING_HUMAN__"}}
 else:
 # Auto-respond using the system LLM
 auto_resp = await self.small_llm.chat.completions.create(
 model=settings.small_model,
 messages=[{"role": "user", "content": agent_prompt}],
 max_tokens=512,
 temperature=0.3,
)
 auto_text = auto_resp.choices[0].message.content

 # Send response back to the MCP agent
```

```

 await session.send_result(
 request_id,
 result={"role": "assistant", "content": {"type":
"text", "text": auto_text}}
)
 return {"role": "assistant", "content": {"type": "text",
"text": auto_text}}

Register the sampling callback with the MCP SDK
session.on_sampling_request(_handle_sampling_request)

3. Handle resumption (waking from a sampling HITL)
if resume_context and resume_context.get("type") ==
"MCP_SAMPLING":
 await session.send_result(
 resume_context["request_id"],
 result={
 "role": "assistant",
 "content": {"type": "text", "text":
resume_context["user_input"]}
 }
)
 yield ProgressEvent(message="Resuming agent with your
response...")

4. Invoke the tool
try:
 result = await session.call_tool(capability_id,
arguments=inputs)
except Exception as e:
 yield ErrorEvent(message=f"Tool call failed: {str(e)}",
retryable=True)
 return

5. Check if sampling interrupt was triggered during tool
execution
if sampling_result_holder["interrupt_needed"]:
 yield SamplingInterruptEvent(
 agent_prompt=sampling_result_holder["interrupt_payload"]
["agent_prompt"],
 reason=sampling_result_holder["interrupt_payload"]
["reason"],
 resume_context={
 "type": "MCP_SAMPLING",
 "request_id":
sampling_result_holder["interrupt_payload"]["request_id"],
 "capability_id": capability_id,
 "inputs": inputs,
 }
)

```

```

)
 return

6. Yield result
for content_block in (result.content or []):
 if content_block.type == "text":
 yield ProgressEvent(message=content_block.text[:200])

 yield FinalResultEvent(
 output={"content": result.content, "is_error":
result.isError},
 success=not result.isError,
)

async def resume(
 self,
 resume_context: dict,
 user_response: str,
 auth_context: "AuthContext",
) -> AsyncIterator[AgentEvent]:
 """
 Resume after a sampling HITL interrupt.
 Re-establishes the session and sends the user's answer to the
agent.
 """
 transport = resume_context["transport"]
 updated_resume = {**resume_context, "user_input": user_response}

 if transport["type"] == "SSE":
 async with sse_client(transport["endpoint"],
headers=auth_context.headers) as (r, w):
 async with ClientSession(r, w) as session:
 async for event in self._execute_session(
 session,
 resume_context["inputs"],
 resume_context["capability_id"],
 state=None,
 resume_context=updated_resume,
):
 yield event

 async def cancel(self, agent_session_id: str) -> None:
 await self.pool.close_session(agent_session_id)

Additional event type for MCP sampling interrupts
@dataclass
class SamplingInterruptEvent(AgentEvent):
 """MCP agent requested human input via sampling/createMessage."""

```

```
agent_prompt: str
reason: str
resume_context: dict
```

### ### 9.3 A2A Handler (Complex: Multi-Turn Agent Conversation)

A2A agents are autonomous. They can run for minutes, ask clarifying questions, send progress updates, and produce artifacts. The handler manages the full A2A task lifecycle.

```
```python
```

```
# superagent/handlers/a2a_handler.py
```

```
class A2AHandler(AgentHandler):
```

```
    """
```

```
    Handles A2A (Agent-to-Agent) protocol calls.
```

```
    A2A agents maintain internal state across a task. They communicate
    via a message/send endpoint and produce a task object with states:
    submitted → working → (input-required)? → completed / failed
```

```
    Key challenge: A2A agents can ask clarifying questions mid-task.
    These must be surfaced to the user as HITL interrupts and the
    task must be resumed with the user's answer.
```

```
    """
```

```
    def __init__(self, http_client):
        self.http = http_client
```

```
    async def invoke(
        self,
        task_or_inputs: dict | str,    # Natural language task description
for A2A
        resolved: "ResolvedAgent",
        auth_context: "AuthContext",
        state: "AgentState",
    ) -> AsyncIterator[AgentEvent]:
```

```
        endpoint = resolved.transport_endpoint
        context_id = str(uuid.uuid4())    # Unique conversation context
```

```
        # Build A2A message payload
```

```
        payload = {
            "jsonrpc": "2.0",
            "id": str(uuid.uuid4()),
            "method": "message/send",
            "params": {
                "message": {
```

```

        "role": "user",
        "parts": [{"type": "text", "text":
str(task_or_inputs)}}],
        "context_id": context_id,
    }
}

# Send the initial task
response = await self.http.post(
    endpoint,
    json=payload,
    headers=auth_context.headers,
)
result = response.json()["result"]

# Extract task_id from response
task_id = result.get("task", {}).get("id") or result.get("id")

# Poll the task until terminal state or interrupt
async for event in self._poll_task(task_id, context_id, endpoint,
auth_context):
    yield event

async def _poll_task(
    self,
    task_id: str,
    context_id: str,
    endpoint: str,
    auth_context: "AuthContext",
    poll_interval: float = 1.0,
) -> AsyncIterator[AgentEvent]:
    """
    Polls A2A task state machine until completion or interrupt.

    A2A task states:
    submitted → working → completed (happy path)
    submitted → working → input-required → (resumed) → working →
completed
    submitted → working → failed
    submitted → working → canceled
    """

    while True:
        status_response = await self.http.get(
            f"{endpoint}/tasks/{task_id}",
            headers=auth_context.headers,
        )
        status = status_response.json()

```

```

task_state = status.get("status", {}).get("state")

if task_state == "working":
    # Stream any progress messages
    for log_msg in status.get("status", {}).get("message",
[]):
        yield ProgressEvent(message=log_msg.get("text",
"Working..."))
    await asyncio.sleep(poll_interval)
    continue

elif task_state == "input-required":
    # Agent is asking a clarifying question
    # Surface this as a ClarifyEvent – Execution Middleware
    # will convert it to a HITL interrupt
    question_parts = status.get("status", {}).get("message",
[])
    question = " ".join(p.get("text", "") for p in
question_parts)

    yield ClarifyEvent(
        question=question,
        options=[], # A2A doesn't typically provide options
        resume_context={
            "task_id": task_id,
            "context_id": context_id,
            "endpoint": endpoint,
            "auth_headers": auth_context.headers,
        }
    )
    return # Suspend – handler will be resumed via resume()

elif task_state == "completed":
    # Collect artifacts
    for artifact in status.get("artifacts", []):
        yield ArtifactEvent(artifact_ref=ArtifactRef(
            artifact_id=str(uuid.uuid4()),
            artifact_type=_infer_type(artifact.get("mimeType")),
            url=artifact.get("uri"),
            mime_type=artifact.get("mimeType"),
            filename=artifact.get("name"),
            session_id="", # Filled in by middleware
        ))

    # Extract result from last message
    messages = status.get("history", [])
    result_text = ""
    if messages:

```

```

        last = messages[-1]
        result_text = " ".join(
            p.get("text", "") for p in last.get("parts", [])
            if p.get("type") == "text"
        )

        yield FinalResultEvent(
            output={"text_result": result_text, "raw_status":
status},

            success=True,
        )
        return

    elif task_state in ("failed", "canceled"):
        error_msg = status.get("status", {}).get("message", "Agent
task failed")

        if isinstance(error_msg, list):
            error_msg = " ".join(p.get("text", "") for p in
error_msg)

        yield ErrorEvent(
            message=str(error_msg),
            retryable=(task_state != "canceled"),
        )
        return

    async def resume(
        self,
        resume_context: dict,
        user_response: str,
        auth_context: "AuthContext",
    ) -> AsyncIterator[AgentEvent]:
        """Resume after user provided input to agent's clarifying
question."""

        task_id = resume_context["task_id"]
        context_id = resume_context["context_id"]
        endpoint = resume_context["endpoint"]

        # Send user's answer back to the agent
        payload = {
            "jsonrpc": "2.0",
            "id": str(uuid.uuid4()),
            "method": "message/send",
            "params": {
                "message": {
                    "role": "user",
                    "parts": [{"type": "text", "text": user_response}],
                    "context_id": context_id,
                    "task_id": task_id,

```

```

        }
    }
}

await self.http.post(endpoint, json=payload,
headers=auth_context.headers)

# Continue polling
async for event in self._poll_task(task_id, context_id, endpoint,
auth_context):
    yield event

```

10. Interrupt System & HITL

10.1 Interrupt Taxonomy

INTERRUPT TYPES

Type	Source	User sees
clarify	Orchestrator LLM	"Before I proceed: [question]"
approval	Middleware	"I'm about to [action]. Proceed?"
auth	Pre-Flight Checker	"Please authorize [service]" + OAuth URL
agent_question	A2A/ACP Agent	"[Agent name] is asking: [question]"

10.2 How LangGraph Handles Interrupts

LangGraph's `interrupt()` function (0.3+) is the cleanest mechanism. When called inside any node, it:

1. Saves the full `AgentState` to the Redis checkpointer at the current `thread_id`
2. Suspends graph execution
3. Returns a `GraphInterrupt` object to the caller (the Gateway)
4. Waits for `graph.invoke(Command(resume=value))` to continue

```
# Inside execute_agent_calls_node, when PreFlightManager detects auth
needed:
```

```

async def _handle_auth_interrupt(preflight, tool_call, state):
    """
    Pauses execution and asks the user to complete OAuth.

```

```

    When the user completes OAuth, the gateway calls
    resume_from_interrupt()
    which invokes Command(resume={...}) on the graph.
    LangGraph resumes from this exact line with the user's response.
    """
    interrupt_id = str(uuid.uuid4())

    # interrupt() is the LangGraph 0.3 way to pause and wait
    # The value passed to interrupt() is what the gateway sends to the
    user
    user_response = interrupt({
        "interrupt_id": interrupt_id,
        "type": "auth",
        "message": f"To use {preflight.tool_name}, please authorize
access.",
        "oauth_url": preflight.oauth_url,
        "tool_call_id": agent_call.id,
    })

    # Execution resumes here when user completes OAuth
    # user_response = {"interrupt_id": ..., "value": "auth_completed"}
    # We can now proceed with the original tool call
    return user_response

```

10.3 The execute_agent_calls_node (Complete)

```

# superagent/nodes/execute_agent_calls.py

async def execute_agent_calls_node(
    state: AgentState,
    middleware: "ExecutionMiddleware",
    agent_call_result_store,
) -> dict:
    """
    Executes agent calls from the latest Orchestrator LLM response.

    SEQUENTIAL EXECUTION ONLY: The system prompt enforces one agent call
    per turn. This node processes calls in order, never in parallel.
    Parallel execution is explicitly avoided to prevent the LLM from
    hallucinating inputs that depend on prior call results.

    Call order within a single turn:
    1. system:: calls first (pure state mutations, no I/O)
    2. external agent calls second (one at a time, sequentially)
    """
    last_message = state["messages"][-1]
    # OpenRouter returns tool_calls as a list on the message object
    raw_calls = getattr(last_message, "tool_calls", []) or []

```

```

    if not raw_calls:
        return {}

    # Partition: system tools vs external agent calls
    system_calls = [tc for tc in raw_calls if
tc["name"].startswith("system:")]
    external_calls = [tc for tc in raw_calls if not
tc["name"].startswith("system:")]

    agent_call_messages = []
    updated_checklist = state.get("task_checklist")
    new_artifacts = dict(state.get("artifacts", {}))
    new_results = {}

    # — Phase 1: System tools (sequential, in-process)


---


    # System tools run first because they may update the checklist or
    # state that affects how external calls are described/logged.
    current_state = state
    for tc in system_calls:
        result = await SYSTEM_TOOL_REGISTRY.dispatch(tc["name"],
tc["arguments"], current_state)
        # System tool results go into context verbatim – they are always
short
        agent_call_messages.append({
            "role": "tool",
            "tool_call_id": tc["id"],
            "content": result.content,
        })
        if result.updated_state:
            # Merge state update so subsequent system calls see it
            current_state = {**current_state, **result.updated_state}
            if "task_checklist" in result.updated_state:
                updated_checklist = result.updated_state["task_checklist"]

    # — Phase 2: External agent calls (strictly sequential)


---


    for tc in external_calls:
        agent_call_id = tc["id"]

        # Parse the call to get agent_id and capability
        agent_id, capability_id, protocol = parse_agent_call(
            tc["name"],
            state.get("pnd_candidates", []),
        )

        # Run the full middleware pipeline (InputGuard → PreFlight →
Handler)

```

```

# This is blocking – we await each call before processing the next
call_result = await middleware.execute(
    agent_call_id=agent_call_id,
    agent_id=agent_id,
    capability_id=capability_id,
    protocol=protocol,
    inputs=tc["arguments"],
    state=current_state,
    stream_callback=lambda msg: _forward_progress_to_stream(
        msg, state["session_id"]
    ),
)

# Text/JSON responses go verbatim into context
# Binary artifacts are registered by reference
if call_result.is_artifact:
    for artifact in call_result.artifacts:
        artifact.session_id = state["session_id"]
        new_artifacts[artifact.artifact_id] = artifact
        context_content = (
            f"[ARTIFACT PRODUCED] Type: {'', ' '.join(a.artifact_type for a in call_result.artifacts)}. "
            f"FileNames: {'', ' '.join(a.filename or 'unnamed' for a in call_result.artifacts)}. "
            f"Use system::read_artifact(artifact_id=...) to access content."
        )
    else:
        # Verbatim text/JSON response – the LLM reasons over the real
data
        context_content = call_result.raw_content

agent_call_messages.append({
    "role": "tool",
    "tool_call_id": agent_call_id,
    "content": context_content,
})

# Auto-update checklist step if one matches this call
updated_checklist = _auto_update_checklist_step(
    checklist=updated_checklist,
    agent_call_id=agent_call_id,
    agent_id=agent_id,
    success=call_result.success,
    result_summary=_extract_brief_summary(context_content),
)

# Store artifact refs (not raw data) in the result store
if call_result.artifacts:

```

```

        new_results[agent_call_id] = {
            "type": "artifacts",
            "artifact_ids": [a.artifact_id for a in
call_result.artifacts],
        }

    return {
        "messages": agent_call_messages,          # Appended via
reducer
        "agent_call_result_store": new_results,    # Merged via
reducer
        "task_checklist": updated_checklist,
        "artifacts": new_artifacts,
    }

```

10.4 Complete Interrupt Flow Diagram

APPROVAL INTERRUPT EXAMPLE

```

User: "Delete all emails from newsletter@spam.com"
|
▼
Orchestrator LLM → agent_call: gmail-mcp-01__delete_emails
|
▼
Execution Middleware
  → InputGuard: PASS
  → PreFlightManager: is_destructive=true → REQUIRE APPROVAL
|
▼
interrupt({type: "approval",
           message: "I'm about to permanently delete all emails from
                    newsletter@spam.com (estimated: 234 emails).
                    This cannot be undone. Shall I proceed?",
           payload: {count: 234, filter: "from:newsletter@spam.com"}})
|
LangGraph saves AgentState to Redis
|
▼
Gateway receives GraphInterrupt → sends approval prompt to user
|
▼
User: "Yes, go ahead"
|
▼
Gateway: graph.invoke(Command(resume={"value": "approved"}))
|
▼

```

```
Graph resumes from interrupt() point
→ interrupt() returns {"value": "approved"}
→ Middleware continues: AuthResolver → MCPHandler
|
▼
Emails deleted → FinalResultEvent
|
▼
OutputNormalizer: "Deleted 234 emails from newsletter@spam.com."
|
▼
LLM context: agent_call_result = "Deleted 234 emails from
newsletter@spam.com."
|
▼
Orchestrator LLM: "Done! I've deleted 234 emails from
newsletter@spam.com."
```

11. Output Normalizer & Data Transformations

11.1 Design Principle: Verbatim Text, Reference for Binaries

Agent text and JSON responses go into the LLM's context **verbatim**. This is how all major production systems work (Claude Desktop, OpenAI Assistants). The LLM needs the real data to reason correctly — a hotel list summary is useless when the LLM needs to pick the cheapest option with a pool.

The only exception is **binary artifacts** (PDFs, images, audio, spreadsheets). These cannot go into a text context window. They are registered by reference, and the LLM reads them explicitly via `system::read_artifact`.

Context window concern: For genuinely large list responses (e.g., 2000 emails as full objects), the `InputGuard` enforces sensible `max_results` / `page_size` defaults when the LLM omits them. This keeps the data genuinely smaller rather than hiding it behind a summary.

```
# superagent/middleware/output_normalizer.py

@dataclass
class AgentCallResult:
    agent_call_id: str
    success: bool
    raw_content: str          # Verbatim text/JSON – goes into LLM context
    is_artifact: bool        # True if response is a binary file
    artifacts: list["ArtifactRef"]
    error_message: Optional[str] = None
```

```

class OutputNormalizer:
    """
    Processes raw agent handler output into an AgentCallResult.

    Text/JSON path: content goes verbatim into messages context.
    Binary artifact path: content registered by reference, description in
    context.
    """

    @staticmethod
    def normalize(raw_output: Any, agent_name: str, agent_call_id: str) ->
AgentCallResult:

        # — Null / empty check


---


        if raw_output is None or raw_output == {} or raw_output == "":
            return AgentCallResult(
                agent_call_id=agent_call_id,
                success=False,
                raw_content=f"{agent_name} returned an empty response.",
                is_artifact=False,
                artifacts=[],
                error_message="Empty response",
            )

        # — Binary artifact detection


---


        # Artifacts are identified by mime_type or explicit artifact
fields
        # in the FinalResultEvent output dict
        artifacts = _extract_artifacts(raw_output, agent_call_id)
        if artifacts:
            return AgentCallResult(
                agent_call_id=agent_call_id,
                success=True,
                raw_content="", # Not used – execute_node builds context
string
                is_artifact=True,
                artifacts=artifacts,
            )

        # — MCP content array path


---


        # MCP tools return {content: [{type: "text", text: "..."}, ...],
isError: bool}
        if isinstance(raw_output, dict) and "content" in raw_output:
            text_parts = [
                block.get("text", "")
                for block in (raw_output.get("content") or [])

```

```

        if block.get("type") == "text"
    ]
    verbatim = "\n".join(text_parts) if text_parts else
str(raw_output)

    return AgentCallResult(
        agent_call_id=agent_call_id,
        success=not raw_output.get("isError", False),
        raw_content=verbatim,
        is_artifact=False,
        artifacts=[],
        error_message=verbatim if raw_output.get("isError") else
None,
    )

# — A2A free-form text path


---


    if isinstance(raw_output, dict) and "text_result" in raw_output:
        return AgentCallResult(
            agent_call_id=agent_call_id,
            success=True,
            raw_content=raw_output["text_result"],
            is_artifact=False,
            artifacts=[],
        )

# — Fallback: stringify


---


    return AgentCallResult(
        agent_call_id=agent_call_id,
        success=True,
        raw_content=str(raw_output),
        is_artifact=False,
        artifacts=[],
    )

def _extract_artifacts(raw_output: Any, agent_call_id: str) ->
list[ArtifactRef]:
    """
    Detects binary artifact payloads in agent output.
    Returns ArtifactRef list if artifacts found, empty list otherwise.
    """
    if not isinstance(raw_output, dict):
        return []

    refs = []

    # A2A artifact format: {"artifacts": [{uri, mimeType, name, ...}]}

```

```

for artifact in raw_output.get("artifacts", []):
    mime = artifact.get("mimeType", "")
    artifact_type = _mime_to_type(mime)
    refs.append(ArtifactRef(
        artifact_id=str(uuid.uuid4()),
        artifact_type=artifact_type,
        session_id="", # Filled in by execute_agent_calls_node
        url=artifact.get("uri"),
        mime_type=mime,
        filename=artifact.get("name"),
    ))

# MCP resource/blob format: {content: [{type: "blob", ...}]}
for block in raw_output.get("content", []):
    if block.get("type") in ("blob", "image"):
        mime = block.get("mimeType", "application/octet-stream")
        refs.append(ArtifactRef(
            artifact_id=str(uuid.uuid4()),
            artifact_type=_mime_to_type(mime),
            session_id="",
            inline_b64=block.get("data"),
            mime_type=mime,
        ))

return refs

def _mime_to_type(mime: str) -> str:
    if "pdf" in mime: return "pdf"
    if "image" in mime: return "image"
    if "audio" in mime: return "audio"
    if "spreadsheet" in mime or "excel" in mime: return "spreadsheet"
    if "text" in mime: return "text"
    if "json" in mime: return "json"
    return "document"

```

11.2 Artifact Handling via `system::read_artifact`

When an agent produces a binary file, `execute_agent_calls_node` registers it in `state["artifacts"]` and injects this description into the LLM's context:

```
[ARTIFACT PRODUCED] Type: pdf. Filename: quarterly_report.pdf.
Use system::read_artifact(artifact_id="<id>") to access content.
```

The `system::read_artifact` tool then extracts content on demand:

```
# superagent/system_tools/artifacts.py
```

```

async def system_read_artifact(artifact_id: str, state: AgentState) ->
str:
    """Extracts readable content from a registered session artifact."""
    artifact = state["artifacts"].get(artifact_id)
    if not artifact:
        return f"Artifact '{artifact_id}' not found in this session."

    match artifact.artifact_type:
        case "pdf":
            content = await _fetch_and_extract_pdf(artifact.url or
artifact.inline_b64)
            return f"PDF content ({artifact.filename}):\\n{content[:5000]}"
        case "image":
            # Return URL for vision-capable models; base64 if inline
            if artifact.inline_b64:
                return f"[IMAGE BASE64 available for vision model –
{artifact.filename}]"
            return f"Image URL: {artifact.url}"
        case "spreadsheet":
            content = await _fetch_and_parse_spreadsheet(artifact.url)
            return f"Spreadsheet ({artifact.filename}) – first 50
rows:\\n{content[:3000]}"
        case "text" | "json":
            if artifact.inline_b64:
                return base64.b64decode(artifact.inline_b64).decode("utf-
8", errors="replace")
            return await _fetch_url_text(artifact.url)
        case _:
            return f"Artifact of type '{artifact.artifact_type}' at:
{artifact.url}"

```

12. System Tool Registry

System tools are always available to the Orchestrator LLM — they are never retrieved from PnD. They are pure Python functions (or thin wrappers around well-known trusted MCP servers) that execute inside the SuperAgent process itself.

12.1 Registration Pattern

```

# superagent/system_tools/registry.py

from dataclasses import dataclass
from typing import Callable, Awaitable, Any

@dataclass
class SystemToolResult:

```

```

        content: str # Goes verbatim into LLM context
        updated_state: Optional[dict] # State fields to merge (e.g.
task_checklist)

@dataclass
class SystemToolSpec:
    name: str # "system::create_checklist"
    description: str
    parameters: dict # OpenAI-format parameters JSON Schema
    handler: Callable # async fn(arguments: dict, state:
AgentState) -> SystemToolResult

class SystemToolRegistry:
    """
    Central registry for all system tools.
    Thread-safe singleton – built once at service startup.
    """
    _tools: dict[str, SystemToolSpec] = {}

    @classmethod
    def register(cls, spec: SystemToolSpec) -> None:
        cls._tools[spec.name] = spec

    @classmethod
    def get_all_schemas(cls) -> list[dict]:
        """Returns OpenAI function-calling format schemas for all
registered tools."""
        return [
            {
                "type": "function",
                "function": {
                    "name": spec.name,
                    "description": spec.description,
                    "parameters": spec.parameters,
                }
            }
            for spec in cls._tools.values()
        ]

    @classmethod
    async def dispatch(
        cls,
        tool_name: str,
        arguments: dict,
        state: "AgentState",
    ) -> SystemToolResult:
        spec = cls._tools.get(tool_name)
        if not spec:

```

```

        return SystemToolResult(
            content=f"Unknown system tool: {tool_name}",
            updated_state=None,
        )
    return await spec.handler(arguments, state)

```

12.2 Registration at Startup

```

# superagent/system_tools/__init__.py
# Called once in main.py during app startup

def register_all_system_tools():
    from .checklist import (
        create_checklist_handler, update_checklist_step_handler,
        add_checklist_step_handler, update_checklist_scope_handler,
        abandon_checklist_handler,
    )
    from .artifacts import read_artifact_handler
    from .workflow import save_as_workflow_template_handler
    from .memory import store_to_memory_handler, query_memory_handler
    from .utils import web_search_handler, get_datetime_handler,
    send_notification_handler

    specs = [
        SystemToolSpec(
            name="system::create_checklist",
            description="Create a task checklist for a multi-step goal.
            Call when starting any task with 3+ steps.",
            parameters={
                "type": "object",
                "properties": {
                    "goal": {"type": "string"},
                    "steps": {"type": "array", "items": {"type":
"string"}}},
                },
            "required": ["goal", "steps"]
        },
        handler=create_checklist_handler,
    ),
        SystemToolSpec(
            name="system::update_checklist_step",
            description="Mark a checklist step as done, failed, or
            skipped. Call after each step completes.",
            parameters={
                "type": "object",
                "properties": {
                    "step_id": {"type": "integer"},
                    "status": {"type": "string", "enum": ["done",
"failed", "skipped"]},
                },
            ),
        ],
    )

```

```

        "result_summary": {"type": "string", "description":
"Brief note on what was accomplished"},
    },
    "required": ["step_id", "status"]
},
handler=update_checklist_step_handler,
),
SystemToolSpec(
    name="system::add_checklist_step",
    description="Add a new step to the active checklist when task
scope expands.",
    parameters={
        "type": "object",
        "properties": {
            "description": {"type": "string"},
            "insert_after": {"type": "integer"},
        },
        "required": ["description"]
    },
    handler=add_checklist_step_handler,
),
SystemToolSpec(
    name="system::update_checklist_scope",
    description="Update checklist goal and reset specified steps
when user changes requirements.",
    parameters={
        "type": "object",
        "properties": {
            "update_description": {"type": "string"},
            "steps_to_redo": {"type": "array", "items": {"type":
"integer"}}},
        "required": ["update_description", "steps_to_redo"]
    },
    handler=update_checklist_scope_handler,
),
SystemToolSpec(
    name="system::abandon_checklist",
    description="Mark the current task as abandoned.",
    parameters={"type": "object", "properties": {"reason":
{"type": "string"}}},
    handler=abandon_checklist_handler,
),
SystemToolSpec(
    name="system::read_artifact",
    description="Read the content of a binary artifact (PDF,
image, spreadsheet) produced in this session.",
    parameters={
        "type": "object",

```

```

        "properties": {"artifact_id": {"type": "string"}},
        "required": ["artifact_id"]
    },
    handler=read_artifact_handler,
),
SystemToolSpec(
    name="system::save_as_workflow_template",
    description="Save the completed workflow as a reusable
template, optionally with a cron schedule.",
    parameters={
        "type": "object",
        "properties": {
            "name": {"type": "string"},
            "schedule_cron": {"type": "string", "description":
"Optional cron expression e.g. '0 9 * * MON'"},
        },
        "required": ["name"]
    },
    handler=save_as_workflow_template_handler,
),
SystemToolSpec(
    name="system::web_search",
    description="Search the web for current information.",
    parameters={
        "type": "object",
        "properties": {"query": {"type": "string"}},
        "required": ["query"]
    },
    handler=web_search_handler,
),
SystemToolSpec(
    name="system::get_current_datetime",
    description="Get the current date and time in ISO 8601
format.",
    parameters={"type": "object", "properties": {}},
    handler=get_datetime_handler,
),
SystemToolSpec(
    name="system::send_notification",
    description="Send a push notification to the user.",
    parameters={
        "type": "object",
        "properties": {"message": {"type": "string"}},
        "required": ["message"]
    },
    handler=send_notification_handler,
),
SystemToolSpec(
    name="system::store_to_memory",

```

```

        description="Save a fact or preference to persistent user
memory.",
        parameters={
            "type": "object",
            "properties": {
                "key": {"type": "string"},
                "value": {"type": "string"},
            },
            "required": ["key", "value"]
        },
        handler=store_to_memory_handler,
    ),
    SystemToolSpec(
        name="system::query_memory",
        description="Retrieve stored facts from user memory.",
        parameters={
            "type": "object",
            "properties": {"query": {"type": "string"}},
            "required": ["query"]
        },
        handler=query_memory_handler,
    ),
]

for spec in specs:
    SystemToolRegistry.register(spec)

```

12.3 Always-On MCP Tools (System MCP Registry)

Some MCP servers are trusted, always-available infrastructure tools (e.g. a web-search MCP, a file-reader MCP you operate yourself). These are registered as system tools that wrap MCP calls internally but appear to the LLM the same as any other system tool — no PnD lookup needed, no auth interrupt possible (credentials are service-level, not user-level).

```

# In system_tools/utils.py

async def web_search_handler(arguments: dict, state: AgentState) ->
SystemToolResult:
    """Wraps an internal web-search MCP server. Always available."""
    query = arguments["query"]
    # Direct MCP call using a service-level connection (not user auth)
    result = await SYSTEM_MCP_CLIENT.call_tool("web_search", {"query":
query})
    return SystemToolResult(content=result.content[0].text,
updated_state=None)

```

13. Task Checklist & Workflow Memory

The Task Checklist is the LLM's working memory for multi-step tasks. It is lightweight (a structured list, not a DAG), LLM-authored (not system-generated), and used to prevent drift on long workflows.

13.1 Checklist Lifecycle

```
User: "Read my inbox, classify emails by intent, create Jira tickets for bugs"
|
▼
Orchestrator LLM (turn 1):
  "Let me clarify – should I process all emails or just unread ones?"
|
▼
User: "Just unread from the last 7 days"
|
▼
Orchestrator LLM (turn 2):
  [tool_call: system::create_checklist({
    "goal": "Process unread emails (last 7 days): classify intent, create
    Jira tickets for bugs",
    "steps": [
      "Fetch unread emails from Gmail (last 7 days)",
      "Classify each email by intent: bug_report, feature_request,
      billing, general",
      "For each bug report: create a Jira ticket with title, description,
      priority",
      "Summarize results to user"
    ]
  })]
|
▼
[Steps execute one by one via tool calls]
[After each step, checklist is updated with status + result_summary]
|
▼
[After step 1]:
  Step 1 ✓ "Fetched 34 unread emails from Gmail (last 7 days)"
[After step 2]:
  Step 2 ✓ "Classified 34 emails: 8 bug reports, 12 feature requests, 14
  general"
[After step 3]:
  Step 3 ✓ "Created 8 Jira tickets (BUG-2301 through BUG-2308)"
```

13.2 Checklist System Tools

All checklist tools are registered in `SystemToolRegistry` at startup (see Section 12). The handler implementations live in `superagent/system_tools/checklist.py`. Key design: every mutating handler saves the current checklist snapshot to `checklist_history` before applying changes, enabling undo.

14. Workflow Caching & CronJob Templates

When a user completes a workflow and wants to repeat it (or schedule it), they can request to save it. The system captures the essential workflow structure from the completed task checklist and the tool/agent calls made.

14.1 What Gets Saved

A `WorkflowTemplate` captures: the goal, the sequence of steps, which agents/tools were called for each step, and the user-configurable parameters (like "which mailbox" or "what date range"). It does NOT capture: raw data outputs, sensitive credentials (these are resolved fresh each run), or session-specific IDs.

```
@dataclass
class WorkflowTemplate:
    template_id: str
    user_id: str
    name: str  # User-assigned: "Weekly Bug
Triage"
    description: str
    goal_template: str  # "Process {mailbox} emails
from last {days} days"
    parameters: dict  # {mailbox: "inbox", days: 7} –
user-configurable
    steps: list["WorkflowTemplateStep"]
    agents_used: list[str]  # agent_ids – for health
checking at run time
    created_from_session: str  # session_id for audit
    schedule: Optional["CronSchedule"]  # None = manual trigger only
    created_at: float
    last_run_at: Optional[float] = None
    run_count: int = 0

@dataclass
class WorkflowTemplateStep:
    order: int
    description: str
    tool_or_agent_id: str  # The tool/agent used
    protocol_type: str  # "mcp" | "a2a" | "acp"
    input_template: dict  # Tool inputs with parameter placeholders
like {days}
```

```

        is_reversible: bool

@dataclass
class CronSchedule:
    cron_expression: str          # "0 9 * * MON" = Monday 9am
    timezone: str                 # "Asia/Karachi"
    enabled: bool = True
    next_run_at: Optional[float] = None

```

14.2 Template Capture Flow

```

# System tool: system::save_as_workflow_template

async def system_save_workflow_template(
    name: str,
    schedule_cron: Optional[str],
    state: AgentState,
) -> str:
    """
    Captures the current session's completed workflow as a reusable
    template.
    Called by the LLM when user requests "save this workflow" or "run this
    weekly".
    """

    checklist = state.get("task_checklist")
    if not checklist or checklist.status != "complete":
        return "No completed workflow to save. Please complete a task
        first."

    # Build steps from completed checklist + agent_call_result_store
    steps = []
    for cs in checklist.steps:
        if cs.status == "done" and cs.agent_id and cs.agent_call_id:
            # Reconstruct the input template from the actual tool call
            # Replace session-specific values with parameter placeholders
            input_template = _extract_input_template(
                tool_call_id=cs.tool_call_id,
                state=state,
            )
            steps.append(WorkflowTemplateStep(
                order=cs.id,
                description=cs.description,
                tool_or_agent_id=cs.agent_id,
                protocol_type=_infer_protocol(cs.agent_id),
                input_template=input_template,
                is_reversible=cs.is_reversible,
            ))

```

```

template = WorkflowTemplate(
    template_id=str(uuid.uuid4()),
    user_id=state["user_id"],
    name=name,
    description=checklist.goal,
    goal_template=checklist.goal,
    parameters={}, # User can configure later in UI
    steps=steps,
    agents_used=list(set(s.tool_or_agent_id for s in steps)),
    created_from_session=state["session_id"],
    schedule=CronSchedule(
        cron_expression=schedule_cron,
        timezone="UTC",
    ) if schedule_cron else None,
    created_at=time.time(),
)

# Persist template
await WorkflowTemplateStore.save(template)

return (
    f"Workflow '{name}' saved (ID: {template.template_id}). "
    + (f"Scheduled: {schedule_cron}" if schedule_cron else "Run it manually anytime.")
)

```

14.3 CronJob Execution

```

# scheduler/cron_runner.py

class WorkflowScheduler:
    """
    Runs scheduled workflow templates as new SuperAgent sessions.

    Each cron execution creates a fresh session, injects the template's
    goal as the initial user message, and lets the ReAct loop execute
    the workflow. No special cron-aware code in the graph – it runs
    exactly like a regular user session.
    """

    async def execute_template(self, template: WorkflowTemplate) -> str:
        """Execute a workflow template as a new session."""

        # Pre-flight: verify all agents in template are healthy
        unhealthy = await self._check_template_agents(template)
        if unhealthy:
            await self._notify_user(
                template.user_id,
                f"Scheduled workflow '{template.name}' could not start: "

```

```

        f"agents unavailable: {'', '.join(unhealthy)}"
    )
    return "failed"

# Create a new session with the template's goal
session_id = str(uuid.uuid4())
initial_message = _render_template_message(template)
# e.g. "Process inbox emails from last 7 days. Classify by intent.
#       Create Jira tickets for bug reports. Summarize results."

# Run as a normal SuperAgent session
# The LLM will see the message and proceed with its ReAct loop
async for event in self.session_runner.run_turn(
    session_id=session_id,
    user_message=initial_message,
):
    # Stream events to user's notification channel
    if event.type in ("progress", "result"):
        await self._send_notification(template.user_id, event)

# Update template metadata
template.last_run_at = time.time()
template.run_count += 1
await WorkflowTemplateStore.update(template)

return "completed"

```

15. Conversational Interface & Context Management

15.1 Preventing Context Window Overwhelm

The LLM's context window is the most fragile resource in a long-running session. A session with 20+ tool calls can easily exceed 100K tokens without careful management. The Context Window Manager compresses old tool results proactively.

```

# superagent/context/window_manager.py

class ContextWindowManager:

    TOKEN_BUDGET = 140_000          # Leave 60K for response + tools
    COMPRESSION_THRESHOLD = 0.85    # Only compress when truly pressing
    (85%)

    # Default pagination limits injected when LLM omits them
    DEFAULT_PAGE_SIZES = {
        "fetch_emails": {"max_results": 50},

```

```

        "list_issues": {"per_page": 30},
        "search_results": {"limit": 20},
        "get_transactions": {"limit": 50},
    }

    @staticmethod
    def compress_if_needed(
        messages: list,
        token_budget: int,
    ) -> list:
        """
        Primary defence against context overflow is pagination (see
        InputGuard).
        Compression here is a last-resort safety net, not a design
        assumption.

        When compression triggers:
        - Keep last 15 messages intact (recent context is critical)
        - For older ToolMessages > 2000 chars: replace with a brief lead-
in
        (first 200 chars + "[...truncated, full response in earlier
        context]")
        - Never compress SystemMessages
        - Never compress messages from last 15
        """
        estimated = _estimate_tokens(messages)

        if estimated < token_budget *
ContextWindowManager.COMPRESSION_THRESHOLD:
            return messages

        recent = messages[-15:]
        older = messages[:-15]

        compressed = []
        for msg in older:
            if (
                isinstance(msg, dict)
                and msg.get("role") == "tool"
                and len(str(msg.get("content", ""))) > 2000
            ):
                content = str(msg["content"])
                compressed.append({
                    **msg,
                    "content": content[:200] + "\n[...earlier response
truncated]"
                })
            else:
                compressed.append(msg)

```

```

        return compressed + recent

    @staticmethod
    def enforce_pagination(tool_name: str, inputs: dict) -> dict:
        """
        Called by InputGuard: injects default page_size/max_results when
        absent.
        Prevents accidentally fetching 10,000 results in one call.
        """
        # Find matching default by tool_name suffix (e.g. gmail-mcp-
        01__fetch_emails)
        for suffix, defaults in
ContextWindowManager.DEFAULT_PAGE_SIZES.items():
            if tool_name.endswith(f"__{suffix}"):
                for param, default_val in defaults.items():
                    if param not in inputs:
                        inputs[param] = default_val
        return inputs

```

15.2 Conversational vs. Task Mode

The system must fluidly handle both pure conversation and task execution within the same session. The LLM governs this naturally — it decides whether a user message requires tool calls or just a conversational response.

The session never "enters task mode" as a state machine. Instead, the presence of an active `task_checklist` in state serves as a soft signal. The system prompt includes the checklist when active, giving the LLM context about in-progress work. But the LLM can still answer off-topic questions without affecting the checklist.

EXAMPLE SESSION (Task + Conversation mixed):

Turn 1: "Analyze my GitHub repo's open issues and create a priority report"

→ LLM: Creates checklist, starts calling GitHub MCP tools

Turn 2: "What's a good way to prioritize bugs vs feature requests?"
(generic question)

→ LLM: Answers directly from knowledge. Checklist unchanged. No tool calls.

Turn 3: "Ok continue with the analysis"

→ LLM: Checks checklist, resumes from where it left off.

Turn 4: "Actually, only look at issues from the last 30 days"

→ LLM: Updates checklist step 1's scope. Re-runs step 1 with new filter.
(See Section 16 for how checklist mutations work)

16. Checklist Mutations & Undo

16.1 Updating the Task Scope Mid-Workflow

When a user changes their requirements mid-workflow, the LLM handles this through natural reasoning — it doesn't require a ManifestDiffEngine. The LLM assesses what needs to change and acts accordingly.

```
# system::update_checklist_scope – called when user changes scope mid-task
```

```
async def system_update_checklist_scope(
    update_description: str, # "Only emails from last 7 days"
    steps_to_redo: list[int], # [1, 2] – step IDs to re-run
    state: AgentState,
) -> dict:
    """
    Updates the checklist goal and marks specified steps as pending.
    The LLM calls this when it determines that scope has changed.

    Completed steps that the LLM does NOT include in steps_to_redo
    are preserved – no unnecessary re-execution.
    """

    checklist = state["task_checklist"]

    # Save current state to undo history BEFORE modifying
    history_entry = (time.time(), copy.deepcopy(checklist))

    # Update goal description
    updated_goal = f"{checklist.goal} [{update_description}]"

    # Mark specified steps as pending for re-execution
    updated_steps = []
    for step in checklist.steps:
        if step.id in steps_to_redo:
            updated_steps.append(ChecklistStep(
                id=step.id,
                description=step.description,
                status="pending", # Reset to pending
                result_summary=None, # Clear old result
                result_ref=None,
                agent_id=None,
            ))
        else:
            updated_steps.append(step) # Keep completed steps as-is

    updated_checklist = TaskChecklist(
```

```

        checklist_id=checklist.checklist_id,
        goal=updated_goal,
        steps=updated_steps,
        status="active",
        created_at=checklist.created_at,
        agents_used=checklist.agents_used,
    )

    return {
        "task_checklist": updated_checklist,
        "checklist_history": [history_entry],    # Appended via reducer
    }

```

16.2 Undo Mechanism

Undo operates on two levels: undo the **checklist state** (restore a previous step configuration) and, for reversible actions, undo the **real-world action** (cancel a booking, delete a ticket, un-send an email if possible).

```

# system::undo_last_action – called by LLM when user says "undo that"

async def system_undo_last_action(state: AgentState, small_llm) -> dict:
    """
    Attempts to reverse the most recently completed reversible action.

    Step 1: Restore previous checklist state from undo history
    Step 2: If the last completed step was reversible, generate a
            reversal tool call and execute it
    """

    history = state.get("checklist_history", [])
    if not history:
        return {"messages": [SystemMessage("Nothing to undo in this
session.")]}

    # Pop the most recent history snapshot
    prev_timestamp, prev_checklist = history[-1]

    # Find the last completed step that was reversible
    current_checklist = state.get("task_checklist")
    last_reversible = None

    if current_checklist:
        for step in reversed(current_checklist.steps):
            if step.status == "done" and step.is_reversible and
step.undo_context:
                last_reversible = step
                break

```

```

undo_instructions = ""
if last_reversible:
    # Ask small LLM to generate reversal instructions
    reversal = await small_llm.ainvoke([
        SystemMessage("Generate a one-sentence instruction to undo
this action. Be specific with all IDs and references."),
        HumanMessage(
            f"Action: {last_reversible.description}\n"
            f"Context: {json.dumps(last_reversible.undo_context)}"
        )
    ])
    undo_instructions = reversal.content

return {
    "task_checklist": prev_checklist,          # Restore previous
state
    "checklist_history": history[:-1],        # Pop from history
    "messages": [SystemMessage(
        f"Checklist restored to previous state."
        + (f" To reverse the real-world action: {undo_instructions}"
if undo_instructions else ""))
    ])
}

```

17. Conditional Routing & LLM Decision-Making

Unlike the old DAG model where conditional routing required explicit Router nodes with evaluated expressions, the ReAct model handles conditionals implicitly through the LLM's reasoning. However, it is important to understand exactly how this works and where to be careful.

17.1 How the LLM Routes Conditionally

Consider the email triage example: "If it's a bug report, create a Jira ticket. Otherwise, generate a reply."

The LLM handles this by calling the classification tool, receiving the result, and then **deciding in its next reasoning step** which tool to call next based on the result. This is natural language conditional reasoning:

```

LLM Turn 1: [tool_call: gmail_mcp__fetch_emails({query: "is:unread"})]
LLM Turn 2: (receives 34 emails)
            [tool_call: system::create_checklist(goal="...", steps=[...])]
            [tool_call: llm_classify_batch(emails=..., categories=[...])]
            (LLM does classification internally – no external tool)

```

```

LLM Turn 3: (receives classification results: 8 bugs, 26 others)
    "I'll process the bug reports first."
    [tool_call: jira_mcp__create_issue({title: "Login fails on
mobile", ...})]
    [tool_call: jira_mcp__create_issue({title: "API timeout on
checkout", ...})]
    ... (8 sequential calls, one per turn)

LLM Turn 4: (receives Jira creation results)
    "Now I'll generate replies for the other 26 emails."
    [tool_call: system::update_checklist_step({step_id: 3, status:
"done", ...})]
    [LLM generates reply text directly – no tool needed]
    [tool_call: gmail_mcp__send_email({to: ..., body: "..."})]
    ... (batch)

```

The key insight is that **there is no special conditional routing mechanism** — the LLM reads tool results and makes the next decision. This works reliably for well-defined conditions. For complex multi-condition branching, the Task Checklist anchors the LLM: it knows the plan and checks off steps systematically.

17.2 Where Conditional Routing Can Fail (and Mitigations)

The LLM may "forget" a condition if the relevant context scrolls out of its working window, or may misroute if a tool result is ambiguous. Two mitigations:

Mitigation 1: The Task Checklist's step descriptions should encode the condition explicitly: "Step 3: For each email classified as 'bug_report', create a Jira ticket." Not just "Create Jira tickets."

Mitigation 2: For high-stakes conditional decisions (e.g., "if account balance > \$10,000 then escalate"), the system prompt instructs the LLM to surface these as **clarify** interrupts rather than assuming: "For financial thresholds or irreversible conditional actions, confirm the condition result with the user before proceeding."

18. Artifact Handling

Artifact handling is covered comprehensively in:

- **Section 11.2** — `OutputNormalizer` detects binary payloads (PDFs, images, spreadsheets, audio) in agent responses and registers them as `ArtifactRef` objects rather than injecting binary data into the LLM's context.
- **Section 12** — `system::read_artifact` in the System Tool Registry lets the LLM read artifact content on demand (PDF text extraction, spreadsheet parsing, etc.).
- **Section 5** — `AgentState.artifacts` dict (keyed by `artifact_id`) persists all artifact references across the session lifetime.

The `ArtifactRef` dataclass carries: `artifact_id`, `artifact_type` (`pdf` | `image` | `audio` | `spreadsheet` | `document` | `url` | `text` | `json`), `session_id`, `url`, `inline_b64` (for small artifacts), `mime_type`, `filename`, `size_bytes`.

When an agent produces an artifact, `execute_agent_calls_node` (Section 10.3):

1. Calls `OutputNormalizer` which detects the binary payload and returns `is_artifact=True`
2. Registers each `ArtifactRef` in `state["artifacts"]`
3. Injects this description into the LLM context: `[ARTIFACT PRODUCED] Type: pdf. Filename: report.pdf. Use system::read_artifact(artifact_id="<id>") to access.`

19. Pre-Flight Checks (Full Auth Resolution)

Pre-flight runs inside `ExecutionMiddleware.execute()` for every external agent call, before the handler is invoked. Auth resolution is **part of pre-flight** — the `PreFlightManager` attempts all configured auth strategies and only raises an interrupt if none can be satisfied without user input. This mirrors the architecture formalised in the original Orchestration Runtime v2.1 spec.

Pre-Flight sequence:

1. Fetch manifest (`ManifestCache`, 5-min TTL)
2. Health check (Redis-cached, 30-sec TTL)
3. Auth resolution (attempt all strategies, HITL only if all fail)
4. Input schema validation (`jsonschema`)
5. Destructiveness check → raise `ApprovalInterrupt` if flagged
6. Return `ExecutionContext` with injected auth headers

19.1 PreFlightManager

```
# superagent/middleware/preflight.py

class PreFlightManager:
    """
    Validates agent readiness and resolves auth before any external call.
    Auth resolution is internal – interrupts are raised only as a last
    resort.
    """

    HEALTH_CACHE_TTL = 30  # seconds

    def __init__(self, manifest_cache, auth_manager, health_cache,
registry):
        self.manifests = manifest_cache
        self.auth = auth_manager
        self.health_cache = health_cache
```

```

self.registry = registry

async def validate(
    self,
    agent_id: str,
    capability_id: Optional[str],
    inputs: dict,
    user_id: str,
) -> "ExecutionContext":
    """
    Runs all pre-flight checks and returns a fully-populated
    ExecutionContext.

    Raises:
        AgentUnreachableError    – health check failed (non-HITL, logged
to LLM)
        AuthRequiredError        – all auth strategies need user action
(→ HITL)
        ApprovalRequiredError    – destructive action needs user
confirmation (→ HITL)
        InputValidationError      – inputs don't match schema (non-HITL,
LLM self-corrects)
    """

    # Step 1: Manifest fetch (cached)
    manifest = await self.manifests.get_manifest(agent_id)

    # Step 2: Health check (cached 30s)
    await self._check_health(agent_id, manifest)

    # Step 3: Auth resolution (full strategy cascade – see 19.2)
    auth_headers, auth_strategy_id = await self.auth.resolve(
        strategies=manifest["security"]["auth_strategies"],
        user_id=user_id,
        agent_id=agent_id,
    )

    # Step 4: Input validation (only for MCP TOOL capabilities)
    if capability_id and manifest.get("protocol_type") == "MCP":
        await self._validate_inputs(manifest, capability_id, inputs)

    # Step 5: Destructiveness check
    capability = self._find_capability(manifest, capability_id)
    is_destructive = capability.get("is_destructive", False) if
capability else False

    # Step 6: Build context
    transport = manifest["transport"]
    return ExecutionContext(

```

```

        agent_id=agent_id,
        agent_name=manifest["name"],
        capability_id=capability_id,
        protocol_type=manifest["protocol_type"],
        transport=transport,
        auth_headers=auth_headers,
        auth_strategy_id=auth_strategy_id,
        manifest=manifest,
        is_destructive=is_destructive,
        timeout_ms=30_000,
        retry_policy={"max_attempts": 3, "backoff": "exponential"},
    )

    async def _check_health(self, agent_id: str, manifest: dict) -> None:
        cache_key = f"health:{agent_id}"
        cached = await self.health_cache.get(cache_key)
        if cached and json.loads(cached).get("healthy"):
            return

        health_url = manifest.get("transport", {}).get("health_endpoint")
        if not health_url:
            return # No health endpoint declared – skip

        try:
            resp = await http_client.get(health_url, timeout=5.0)
            healthy = resp.status_code == 200
        except Exception as e:
            healthy = False
            error = str(e)

        result = {"healthy": healthy, "error": error if not healthy else
None}
        await self.health_cache.set(
            cache_key, json.dumps(result), ex=self.HEALTH_CACHE_TTL
        )

        if not healthy:
            raise AgentUnreachableError(
                f"Agent {agent_id} health check failed. "
                f"Consider requesting an alternative agent."
            )

    async def _validate_inputs(self, manifest: dict, capability_id: str,
inputs: dict) -> None:
        capabilities = manifest.get("capabilities", {}).get("tools", [])
        tool_schema = next(
            (t["input_schema"] for t in capabilities if t["name"] ==
capability_id),
            None

```

```

    )
    if not tool_schema:
        return # No schema to validate against

    try:
        jsonschema.validate(instance=inputs, schema=tool_schema)
    except jsonschema.ValidationError as e:
        raise InputValidationError(
            f"Input validation failed for {capability_id}:
{e.message}"
        )

    def _find_capability(self, manifest: dict, capability_id:
Optional[str]) -> Optional[dict]:
        if not capability_id:
            return None
        for cap in manifest.get("capabilities", {}).get("tools", []):
            if cap["name"] == capability_id:
                return cap
        return None

```

19.2 AuthManager — Full Strategy Cascade

The `AuthManager` attempts each strategy in the order listed in the manifest's `auth_strategies` array. It only raises `AuthRequiredError` (which triggers a HITL interrupt) if all strategies are exhausted without success.

```

# superagent/middleware/auth_manager.py

class AuthManager:
    """
    Resolves auth credentials for an agent call.
    Credentials never appear in LLM context or AgentState messages.
    They are injected into the ExecutionContext auth_headers at the last
    moment.

    Supported strategies (from AuthType enum in Prisma schema):
    X_API_KEY, HTTP_BEARER, OAUTH2, OAUTH2_DCR
    mTLS is handled at transport layer via ExecutionContext.
    """

    def __init__(self, vault, token_cache):
        self.vault = vault
        self.cache = token_cache

    async def resolve(
        self,
        strategies: list[dict],

```

```

        user_id: str,
        agent_id: str,
    ) -> tuple[dict, str]:
        """
        Try each strategy in order. Return (headers, strategy_id) on first
        success.
        Raise AuthRequiredError if none succeed.
        """
        last_error = None

        for strategy in strategies:
            try:
                match strategy["type"]:
                    case "X_API_KEY":
                        return await self._handle_api_key(strategy,
user_id)

                    case "HTTP_BEARER":
                        return await self._handle_bearer(strategy,
user_id)

                    case "OAUTH2":
                        return await self._handle_oauth2(strategy,
user_id)

                    case "OAUTH2_DCR":
                        return await self._handle_oauth2_dcr(strategy,
user_id, agent_id)

                    case _:
                        return ({}, strategy["strategy_id"]) # No auth
needed

            except AuthRequiredError:
                raise # Don't try next strategy – user input is needed
            except Exception as e:
                last_error = e
                continue # Try next strategy

        raise AuthenticationError(
            f"All auth strategies exhausted. Last error: {last_error}"
        )

    async def _handle_api_key(self, strategy: dict, user_id: str) ->
tuple:
        config = strategy["config"]
        key = await self.vault.get_user_secret(user_id,
config["key_vault_ref"])

        if key:
            return ({config["header_name"]: key}, strategy["strategy_id"])

        raise AuthRequiredError(
            interrupt_type="AUTH_FORM_SUBMISSION",

```

```

        resume_context={"vault_key": config["key_vault_ref"],
"strategy_id": strategy["strategy_id"]},
        metadata={
            "form_title": f"API Key Required",
            "field_label": config["header_name"],
            "is_secret": True,
        }
    )

    async def _handle_bearer(self, strategy: dict, user_id: str) -> tuple:
        config = strategy["config"]
        token = await self.vault.get_user_secret(user_id,
config["token_vault_ref"])
        scheme = config.get("scheme", "Bearer")

        if token:
            return ({ "Authorization": f"{scheme} {token}" },
strategy["strategy_id"])

        raise AuthRequiredError(
            interrupt_type="AUTH_FORM_SUBMISSION",
            resume_context={"vault_key": config["token_vault_ref"],
"strategy_id": strategy["strategy_id"]},
            metadata={"form_title": "Bearer Token Required", "is_secret":
True},
        )

    async def _handle_oauth2(self, strategy: dict, user_id: str) -> tuple:
        config = strategy["config"]
        provider = config["provider_hint"]

        # 1. Check cache for access token
        cache_key = f"auth:{user_id}:{provider}:access"
        token = await self.cache.get(cache_key)
        if token:
            return ({ "Authorization": f"Bearer {token}" },
strategy["strategy_id"])

        # 2. Check vault for refresh token → auto-refresh
        refresh = await self.vault.get_user_secret(user_id, f"
{provider}_refresh_token")
        if refresh:
            client_id, client_secret = await
self.vault.get_global_provider_creds(provider)
            resp = await oauth_client.refresh_token(
                token_url=config["token_url"],
                refresh_token=refresh,
                client_id=client_id,
                client_secret=client_secret,

```

```

        )
        await self.cache.set(cache_key, resp["access_token"],
ex=resp["expires_in"])
        if "refresh_token" in resp:
            await self.vault.save_user_secret(user_id, f"
{provider}_refresh_token", resp["refresh_token"])
            return ({"Authorization": f"Bearer {resp['access_token']}"},
strategy["strategy_id"])

    # 3. No tokens → trigger OAuth login HITL
    client_id, _ = await
self.vault.get_global_provider_creds(provider)
    state_param = encode_state({"user_id": user_id, "type": "oauth2",
"provider": provider})
    auth_url = build_authorization_url(
        config["authorization_url"], client_id, CALLBACK_URL,
        " ".join(config["scopes"]), state_param,
    )
    raise AuthRequiredError(
        interrupt_type="AUTH_CALLBACK",
        resume_context={"type": "oauth2", "provider": provider,
"state": state_param},
        metadata={"auth_url": auth_url, "provider_name":
provider.title()},
    )

    async def _handle_oauth2_dcr(self, strategy: dict, user_id: str,
agent_id: str) -> tuple:
        """OAuth with Dynamic Client Registration – agent provides its own
auth server."""
        creds = await self.vault.get_agent_registration(agent_id, user_id)

        if not creds:
            # Register with agent's DCR endpoint
            reg = await http_client.post(
                strategy["config"]["registration_endpoint"],
                json={"redirect_uris": [CALLBACK_URL], "client_name":
f"MetaOrcha/{user_id}"},
            )
            creds = {"client_id": reg["client_id"], "client_secret":
reg["client_secret"]}
            await self.vault.save_agent_registration(agent_id, user_id,
creds)

    # Then follow same cache → refresh → HITL pattern as standard
OAuth2
    cache_key = f"auth:{user_id}:{agent_id}:access"
    token = await self.cache.get(cache_key)
    if token:

```

```

        return ({ "Authorization": f"Bearer {token}" },
strategy["strategy_id"])

    refresh = await self.vault.get_user_secret(user_id, f"
{agent_id}_refresh")
    if refresh:
        resp = await oauth_client.refresh_token(
            strategy["config"]["token_endpoint"], refresh,
            creds["client_id"], creds["client_secret"],
        )
        await self.cache.set(cache_key, resp["access_token"],
ex=resp["expires_in"])
        return ({ "Authorization": f"Bearer {resp['access_token']}" },
strategy["strategy_id"])

    state_param = encode_state({"user_id": user_id, "type":
"oauth2_dcr", "agent_id": agent_id})
    auth_url = build_authorization_url(
        strategy["config"]["authorization_endpoint"],
creds["client_id"],
        CALLBACK_URL, " ".join(strategy["config"]["scopes"]),
state_param,
    )
    raise AuthRequiredError(
        interrupt_type="AUTH_CALLBACK",
        resume_context={"type": "oauth2_dcr", "agent_id": agent_id,
"state": state_param},
        metadata={"auth_url": auth_url},
    )

```

19.3 Pre-Flight Summary Table

Check	When	Cached	On Failure
Manifest fetch	Every call	5-min TTL (ManifestCache)	ValueError → error to LLM
Agent health ping	Every call	30-sec TTL (Redis)	AgentUnreachableError → error to LLM with alt-agent suggestion
API key / Bearer token	Every call	No (vault lookup)	AuthRequiredError → AUTH_FORM_SUBMISSION interrupt
OAuth access token	Every call	Redis, token expiry - 5min	Auto-refresh if refresh token exists; else AUTH_CALLBACK interrupt
OAuth2 DCR registration	First call per agent	Vault (permanent)	Register automatically, then follow OAuth2 flow
Input schema validation	MCP TOOL calls only	Schema cached in manifest	InputValidationError → error message to LLM (self-correct)

Check	When	Cached	On Failure
Destructiveness flag	Every external call	Manifest cache	ApprovalRequiredError → HITL_APPROVAL interrupt

```

Checks if the agent/server is reachable and responsive.
Results are cached for 30 seconds to avoid hammering agents
with health checks on every tool call.
"""
cache_key = f"health:{resolved.agent_id or resolved.server_id}"

# Check cache first
cached = await health_cache.get(cache_key)
if cached:
    return HealthResult(**json.loads(cached))

# Live health check
try:
    if resolved.protocol == "mcp":
        # For MCP: attempt a brief connection and initialize handshake
        result = await _mcp_health_check(resolved.transport)
    elif resolved.protocol in ("a2a", "acp"):
        # For A2A/ACP: check the agent card / well-known endpoint
        result = await _http_health_check(resolved.transport_endpoint)
    else:
        result = HealthResult(healthy=False, error="Unknown protocol")
except Exception as e:
    result = HealthResult(healthy=False, error=str(e))

# Cache the result (even negative results – avoid stampedes)
await health_cache.set(cache_key, json.dumps(result.__dict__),
ex=PreFlightChecker.HEALTH_CACHE_TTL)
return result

@staticmethod
async def _check_auth(resolved, user_id: str) -> "AuthCheckResult":
    """
    Checks whether the user has valid auth for this agent.
    Does NOT fetch the actual credentials (that's AuthResolver's job).
    Just checks if credentials exist and are not expired.

    For OAuth: checks if token exists and is not within 5 minutes of expiry
    For API keys: checks if key exists in vault
    For public agents: always returns auth_ready=True
    """

```

```

auth_strategy = resolved.auth_strategy

if auth_strategy is None or auth_strategy == "public":
    return AuthCheckResult(missing=False)

if auth_strategy == "api_key":
    exists = await VaultClient.check_exists(
        f"user:{user_id}:agent:{resolved.agent_id}:api_key"
    )
    return AuthCheckResult(
        missing=not exists,
        oauth_url=None,
    )

if auth_strategy == "oauth2":
    token_meta = await TokenCache.get_metadata(
        f"oauth:{user_id}:{resolved.agent_id}"
    )
    if not token_meta:
        return AuthCheckResult(
            missing=True,
            oauth_url=resolved.oauth_authorization_url,
        )
    # Check expiry (5-minute buffer)
    if token_meta["expires_at"] < time.time() + 300:
        return AuthCheckResult(
            missing=True,
            oauth_url=resolved.oauth_refresh_url,
        )
    return AuthCheckResult(missing=False)

return AuthCheckResult(missing=False)

```

20. Database Schema Notes

SuperAgent uses the existing **shared Postgres database** (managed via `commons/db/` Prisma schema). It does not own separate tables. Below are the tables it reads from, and two new tables it adds.

20.1 Tables SuperAgent Reads (existing, no changes)

Table	Purpose
<code>agents</code>	Manifest lookup by <code>agent_id</code> (DID). Health status pre-filtered by PnD.

Table	Purpose
transports	endpoint, type, command for MCPHandler / A2AHandler.
security_configs + auth_strategies	Auth strategy cascade in AuthManager.
capabilities	Input schema for InputGuard and MCPHandler.call_tool.
payment_configs	Payment flag, deferred to future payments feature.
users	user_id validation in session creation.
agent_embeddings	Used by PnD (not by SuperAgent directly).

20.2 New Tables Added by SuperAgent

```
-- Persists reusable workflow templates saved by users
CREATE TABLE workflow_templates (
  id TEXT PRIMARY KEY, -- UUID
  user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  description TEXT,
  goal_template TEXT NOT NULL,
  parameters JSONB DEFAULT '{}',
  steps JSONB NOT NULL, --
list[WorkflowTemplateStep]
  agents_used TEXT[] DEFAULT '{}', -- agent_ids for pre-run
health check
  created_from_session TEXT,
  schedule_cron TEXT, -- NULL = manual only
  schedule_tz TEXT DEFAULT 'UTC',
  schedule_enabled BOOLEAN DEFAULT FALSE,
  next_run_at TIMESTAMPTZ,
  last_run_at TIMESTAMPTZ,
  run_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_wft_user ON workflow_templates(user_id);
CREATE INDEX idx_wft_next_run ON workflow_templates(next_run_at) WHERE
schedule_enabled = TRUE;

-- Persists user-level encrypted secrets (API keys, refresh tokens)
-- This IS the Vault – backed by Postgres with AES-256 encryption at the
app layer
CREATE TABLE user_secrets (
  user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  key TEXT NOT NULL, -- e.g. "GOOGLE_MAPS_KEY"
```

```

    encrypted_value BYTEA NOT NULL,                                -- AES-256-GCM, key from
    VAULT_KEY env var
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (user_id, key)
);

-- Per-agent OAuth client registrations (for OAUTH2_DCR strategy)
CREATE TABLE agent_registrations (
    agent_id        TEXT NOT NULL,
    user_id         TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    client_id       TEXT NOT NULL,
    encrypted_client_secret BYTEA NOT NULL,
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (agent_id, user_id)
);

```

20.3 Prisma Model Additions

Add to `commons/db/schema.prisma`:

```

model WorkflowTemplate {
  id          String    @id @default(cuid())
  user_id     String
  name        String
  description  String?   @db.Text
  goal_template String   @db.Text
  parameters  Json       @default("{}")
  steps       Json
  agents_used String[]
  created_from_session String?
  schedule_cron String?
  schedule_tz  String    @default("UTC")
  schedule_enabled Boolean @default(false)
  next_run_at  DateTime?
  last_run_at  DateTime?
  run_count    Int       @default(0)
  created_at   DateTime  @default(now())
  updated_at   DateTime  @updatedAt
  user         User      @relation(fields: [user_id], references:
[id], onDelete: Cascade)
  @@index([user_id])
  @@index([next_run_at])
  @@map("workflow_templates")
}

model UserSecret {
  user_id String
  key      String
  encrypted_value Bytes

```

```

    created_at      DateTime @default(now())
    user            User      @relation(fields: [user_id], references: [id],
onDelete: Cascade)
    @@id([user_id, key])
    @@map("user_secrets")
}

model AgentRegistration {
    agent_id          String
    user_id           String
    client_id         String
    encrypted_client_secret Bytes
    created_at        DateTime @default(now())
    user              User      @relation(fields: [user_id],
references: [id], onDelete: Cascade)
    @@id([agent_id, user_id])
    @@map("agent_registrations")
}

```

21. CLI Interface

The CLI is a **separate process** from the SuperAgent service. It communicates over HTTP with the service's FastAPI server. This allows you to run the service in one terminal and the CLI in another, seeing service logs independently.

21.1 SuperAgent HTTP API (minimal, pre-Gateway)

```

# superagent/api/routes.py

POST /sessions
  Body: { user_id: str }
  Returns: { session_id: str }

POST /sessions/{session_id}/message
  Body: { message: str }
  Returns: SSE stream of events:
    {"type": "progress", "message": "Connecting to Gmail..."}
    {"type": "token",     "content": "I found 34 emails..."} # LLM token
stream
    {"type": "interrupt", "interrupt_type": "AUTH_CALLBACK", "metadata":
{...}}
    {"type": "done",     "response": "Full final response text"}

POST /sessions/{session_id}/resume
  Body: { interrupt_id: str, value: str }
  Returns: SSE stream (same as /message)

```

```
GET /sessions/{session_id}/status
```

```
Returns: { status: "active"|"suspended"|"completed", pending_interrupt: {...}|null }
```

```
GET /health
```

```
Returns: { status: "ok" }
```

21.2 CLI Client

```
#!/usr/bin/env python3
```

```
# cli/chat.py
```

```
"""
```

```
MetaOrcha SuperAgent CLI
```

```
Usage:
```

```
# Terminal 1 – start the service
```

```
cd services/superagent && uv run uvicorn superagent.main:app --reload --port 8001
```

```
# Terminal 2 – chat
```

```
cd services/superagent && uv run python cli/chat.py --port 8001  
"""
```

```
import httpx
```

```
import asyncio
```

```
import sys
```

```
import json
```

```
from typing import Optional
```

```
BASE_URL = "http://localhost:8001"
```

```
async def stream_sse(response: httpx.Response, session_id: str) ->
```

```
Optional[str]:
```

```
    """Stream SSE events, return final response text or interrupt info."""
```

```
    final_response = []
```

```
    async for line in response.aiter_lines():
```

```
        if not line.startswith("data: "):
```

```
            continue
```

```
        try:
```

```
            event = json.loads(line[6:])
```

```
        except json.JSONDecodeError:
```

```
            continue
```

```
        match event.get("type"):
```

```
            case "progress":
```

```
                print(f" 🔄 {event['message']}", flush=True)
```

```

        case "token":
            print(event["content"], end="", flush=True)
            final_response.append(event["content"])

        case "interrupt":
            print(f"\n\n[INTERRUPT: {event['interrupt_type']}]")
            print(f"    {event.get('message', '')}")

            if event["interrupt_type"] == "AUTH_CALLBACK":
                print(f"\n  Open this URL to authorise:
{event['metadata']['auth_url']}")
                print("    Press Enter when authorisation is
complete...")

                input()
                return
            f"__RESUME__{event['interrupt_id']}__auth_completed"

            elif event["interrupt_type"] in ("HITL_APPROVAL",
"AUTH_FORM_SUBMISSION"):
                user_val = input(f"\n  Your response: ")
                return
            f"__RESUME__{event['interrupt_id']}__{user_val}"

        case "done":
            print() # newline after streamed tokens
            return event.get("response", "").join(final_response)

    return "".join(final_response)

async def main():
    import argparse
    parser = argparse.ArgumentParser(description="MetaOrcha SuperAgent
CLI")
    parser.add_argument("--port", default=8001, type=int)
    parser.add_argument("--user", default="dev-user-001")
    args = parser.parse_args()

    base = f"http://localhost:{args.port}"

    async with httpx.AsyncClient(timeout=120.0) as client:
        # Create session
        resp = await client.post(f"{base}/sessions", json={"user_id":
args.user})
        session_id = resp.json()["session_id"]
        print(f"Session: {session_id}")
        print("Type 'exit' to quit, 'status' to check session state.\n")

```

```

while True:
    try:
        user_input = input("You: ").strip()
    except (EOFError, KeyboardInterrupt):
        print("\nBye!")
        break

    if not user_input:
        continue
    if user_input.lower() == "exit":
        break
    if user_input.lower() == "status":
        r = await client.get(f"{base}/sessions/{session_id}/status")
        print(f" {r.json()}")
        continue

    print("Agent: ", end="", flush=True)

    async with client.stream(
        "POST",
        f"{base}/sessions/{session_id}/message",
        json={"message": user_input},
    ) as response:
        result = await stream_sse(response, session_id)

    # Handle resume if an interrupt occurred
    if result and result.startswith("__RESUME__"):
        parts = result.split("__", 3)
        interrupt_id, value = parts[2], parts[3]
        print("Agent: ", end="", flush=True)
        async with client.stream(
            "POST",
            f"{base}/sessions/{session_id}/resume",
            json={"interrupt_id": interrupt_id, "value": value},
        ) as response:
            await stream_sse(response, session_id)

if __name__ == "__main__":
    asyncio.run(main())

```

22. Test Harness & Test Cases

Tests are split 70 / 25 / 5 across unit / integration / e2e. All external agent calls are mocked. The test suite uses `pytest` with `pytest-asyncio` and `respx` for HTTP mocking.

22.1 Shared Fixtures (tests/conftest.py)

```
# tests/conftest.py
import pytest
from unittest.mock import AsyncMock, MagicMock
from langgraph.checkpoint.memory import MemorySaver
from superagent.graph.builder import build_superagent_graph
from superagent.graph.state import AgentState

@pytest.fixture
def mock_orchestrator_llm():
    """Returns a mock LLM that responds with a fixed text response."""
    llm = AsyncMock()
    llm.chat.completions.create = AsyncMock(return_value=_make_text_response(
        "I'll help you with that."
    ))
    return llm

@pytest.fixture
def mock_small_llm():
    llm = AsyncMock()
    llm.chat.completions.create = AsyncMock(return_value=_make_text_response("YES"))
    return llm

@pytest.fixture
def mock_pnd_client():
    client = AsyncMock()
    client.get_candidates = AsyncMock(return_value=_make_pnd_response([
        {"agent_id": "did:emerge:agent:gmail-01", "protocol_type": "MCP",
        "agent_name": "Gmail", "capabilities": [
            {"capability_id": "fetch_emails", "capability_type": "T00L",
            "name": "fetch_emails", "description": "Fetch emails",
            "input_schema": {"type": "object", "properties": {"query":
{"type": "string"}}}}}
    ]})
    return client

@pytest.fixture
def memory_graph(mock_orchestrator_llm, mock_small_llm, mock_pnd_client):
    """Full compiled graph using MemorySaver (no Redis needed for
    unit/integration tests)."""
    graph, _ = build_superagent_graph(
        redis_url="", # Overridden by MemorySaver
        orchestrator_llm=mock_orchestrator_llm,
        small_llm=mock_small_llm,
        pnd_client=mock_pnd_client,
```

```

        execution_middleware=AsyncMock(),
        vault=AsyncMock(),
        agent_call_result_store=AsyncMock(),
        checkpointer_override=MemorySaver(),    # Test override
    )
    return graph

def _make_text_response(text: str):
    """Helper: mock OpenRouter text response."""
    choice = MagicMock()
    choice.message.content = text
    choice.message.tool_calls = None
    resp = MagicMock()
    resp.choices = [choice]
    resp.usage.total_tokens = 50
    return resp

def _make_tool_call_response(tool_name: str, arguments: dict):
    """Helper: mock OpenRouter response with a tool call."""
    tc = MagicMock()
    tc.id = "tc_test_001"
    tc.function.name = tool_name
    tc.function.arguments = json.dumps(arguments)
    choice = MagicMock()
    choice.message.content = None
    choice.message.tool_calls = [tc]
    resp = MagicMock()
    resp.choices = [choice]
    resp.usage.total_tokens = 80
    return resp

```

22.2 Unit Tests

tests/unit/test_input_guard.py

```

# Tests: required field detection, type coercion, constraint violations,
self-correct messages
def test_missing_required_field(): ...
def test_type_mismatch_integer_given_string(): ...
def test_constraint_violation_max_results_exceeded(): ...
def test_valid_inputs_pass(): ...
def test_error_message_is_llm_friendly(): ...

```

tests/unit/test_output_normalizer.py

```

# Tests: MCP content array path, A2A text path, artifact detection, empty
response, fallback
def test_mcp_text_content_returned_verbatim(): ...

```

```
def test_a2a_text_result_returned_verbatim(): ...
def test_binary_artifact_detected_and_registered(): ...
def test_empty_response_returns_error_result(): ...
def test_pdf_artifact_mime_detection(): ...
```

tests/unit/test_pnd_gate.py

```
# Tests: tier-1 NO patterns, tier-1 TOOL patterns, active checklist
bypass,
#         tier-2 embedding score thresholds, tier-3 Haiku fallback
def test_conversational_message_returns_false(): ...
def test_tool_keyword_returns_true(): ...
def test_active_checklist_always_returns_true(): ...
def test_ambiguous_message_uses_haiku(): ...
```

tests/unit/test_auth_manager.py

```
# Tests: API key found, API key missing → interrupt, OAuth cache hit,
#         OAuth refresh success, OAuth refresh expired → interrupt,
#         DCR registration flow, strategy cascade fallback
def test_api_key_found_in_vault(): ...
def test_api_key_missing_raises_auth_required(): ...
def test_oauth2_cache_hit_returns_token(): ...
def test_oauth2_refresh_auto_refreshes(): ...
def test_oauth2_no_tokens_raises_auth_callback(): ...
def test_dcr_registers_then_authenticates(): ...
def test_strategy_cascade_tries_next_on_failure(): ...
```

tests/unit/test_checklist.py

```
# Tests: creation, step updates, scope changes, undo, history preservation
def test_create_checklist_sets_active_status(): ...
def test_update_step_marks_done(): ...
def test_update_scope_resets_specified_steps(): ...
def test_undo_restores_previous_snapshot(): ...
def test_checklist_history_is_append_only(): ...
```

tests/unit/test_preflight.py

```
def test_health_check_cached_after_first_call(): ...
def test_health_check_miss_raises_unreachable(): ...
def test_destructive_capability_raises_approval(): ...
def test_input_schema_validation_catches_mismatch(): ...
```

22.3 Integration Tests

tests/integration/test_mcp_handler.py

```
# Uses a mock MCP server (respx or a real in-process MCP stub)
# Tests: SSE transport, STDIO transport, tool invocation,
#         sampling classifier → auto-respond, sampling classifier → HITL,
#         resume after sampling HITL, error from server

@pytest.mark.asyncio
async def test_basic_tool_call_returns_result():
    """Handler calls tool, receives content, yields FinalResultEvent."""
    ...

@pytest.mark.asyncio
async def test_sampling_auto_response_for_routine_task():
    """Classifier says no human needed → auto-respond, tool completes."""
    ...

@pytest.mark.asyncio
async def test_sampling_triggers_hitl_for_destructive_action():
    """Classifier says human needed → SamplingInterruptEvent yielded."""
    ...

@pytest.mark.asyncio
async def test_resume_after_sampling_hitl():
    """Handler receives resume_context with user_input → sends to agent →
    completes."""
    ...
```

tests/integration/test_a2a_handler.py

```
@pytest.mark.asyncio
async def test_task_polling_happy_path(): ...
@pytest.mark.asyncio
async def test_input_required_yields_clarify_event(): ...
@pytest.mark.asyncio
async def test_resume_after_input_required(): ...
@pytest.mark.asyncio
async def test_task_failed_yields_error_event(): ...
@pytest.mark.asyncio
async def test_artifact_in_completed_task(): ...
```

tests/integration/test_execute_node.py

```
# Tests the full execute_agent_calls_node with mocked middleware
@pytest.mark.asyncio
async def test_system_calls_run_before_external_calls(): ...
@pytest.mark.asyncio
async def test_single_external_call_verbatim_in_context(): ...
```

```
@pytest.mark.asyncio
async def test_artifact_produces_reference_in_context(): ...
@pytest.mark.asyncio
async def test_agent_unavailable_returns_error_to_llm(): ...
```

tests/integration/test_orchestrator_node.py

```
@pytest.mark.asyncio
async def test_pnd_not_called_for_conversational_message(): ...
@pytest.mark.asyncio
async def test_pnd_called_when_tool_keyword_present(): ...
@pytest.mark.asyncio
async def test_active_checklist_triggers_pnd(): ...
@pytest.mark.asyncio
async def test_tool_schemas_injected_in_openrouter_format(): ...
```

tests/integration/test_session_graph.py

```
@pytest.mark.asyncio
async def test_single_turn_text_response(): ...
@pytest.mark.asyncio
async def test_agent_call_loops_back_to_orchestrator(): ...
@pytest.mark.asyncio
async def test_state_persists_across_turns(memory_graph): ...
```

22.4 End-to-End Tests

These use the full graph with `MemorySaver` and mock all external HTTP/MCP calls via `respx`.

tests/e2e/test_single_turn.py

```
@pytest.mark.asyncio
async def test_generic_question_answered_without_agents():
    """'What is the Pythagorean theorem?' → text response, no PnD call."""

@pytest.mark.asyncio
async def test_single_mcp_tool_call_completes():
    """'What is the current ETH price?' → PnD → MCP call → result in
    context → LLM responds."""
```

tests/e2e/test_multi_turn.py

```
@pytest.mark.asyncio
async def test_checklist_created_and_updated_across_turns():
    """3-step task creates checklist on turn 1, updates steps on turns 2-4."""
```

```

@pytest.mark.asyncio
async def test_scope_change_mid_workflow_resets_steps():
    """User changes filter criteria mid-task → checklist step reset, re-
    executed."""

@pytest.mark.asyncio
async def test_conversational_turn_mid_task_doesnt_disrupt_checklist():
    """Off-topic question mid-task → checklist unchanged → task resumes
    correctly."""

```

tests/e2e/test_hitl_auth.py

```

@pytest.mark.asyncio
async def test_missing_api_key_triggers_interrupt():
    """Vault miss → AUTH_FORM_SUBMISSION interrupt raised and
    checkpointed."""

@pytest.mark.asyncio
async def test_resume_after_api_key_submission():
    """After key saved, Command(resume=...) re-runs preflight →
    success."""

@pytest.mark.asyncio
async def test_oauth_callback_flow():
    """AUTH_CALLBACK interrupt → simulate callback → token cached →
    resume."""

```

tests/e2e/test_hitl_approval.py

```

@pytest.mark.asyncio
async def test_destructive_action_triggers_approval_interrupt(): ...
@pytest.mark.asyncio
async def test_user_approves_and_execution_continues(): ...
@pytest.mark.asyncio
async def test_user_declines_and_execution_halts(): ...

```

tests/e2e/test_hitl_agent_question.py

```

@pytest.mark.asyncio
async def test_a2a_input_required_surfaces_to_user(): ...
@pytest.mark.asyncio
async def test_mcp_sampling_hitl_surfaces_to_user(): ...
@pytest.mark.asyncio
async def test_resume_with_user_answer_continues_agent(): ...

```

tests/e2e/test_checklist_mutation.py

```
@pytest.mark.asyncio
async def test_undo_restores_checklist_and_notifies_llm(): ...
@pytest.mark.asyncio
async def test_add_step_mid_workflow(): ...
```

tests/e2e/test_workflow_template.py

```
@pytest.mark.asyncio
async def test_save_workflow_template_after_completion(): ...
@pytest.mark.asyncio
async def test_cron_template_executes_as_new_session(): ...
```

23. Recommended Models for Production

All model calls go through **OpenRouter** (<https://openrouter.ai/api/v1>). Model IDs are configuration values in `.env` — never hardcoded.

COMPONENT	OPENROUTER MODEL ID
RATIONALE	
Orchestrator LLM Primary reasoner.	anthropic/claude-sonnet-4-5
Reliable tool use.	
200K context.	
PnD Gate Tier 3 classifier Binary YES/NO.	anthropic/claude-haiku-4-5
~50 tokens in/out.	
Every turn, cost-critical.	
MCP Sampling classifier Classify needs_human.	anthropic/claude-haiku-4-5
JSON output, low tokens.	
MCP Sampling auto-responder Routine LLM tasks	anthropic/claude-haiku-4-5

requested by agents.

Undo instruction generator | anthropic/claude-haiku-4-5 |
One sentence extraction.

Fallback / cost optimisation | meta-llama/llama-3.3-70b-instruct |
Optional swap-in for
small_model tasks.

```
# superagent/config.py – all model refs are config, never hardcoded
class Settings(BaseSettings):
    orchestrator_model: str = "anthropic/claude-sonnet-4-5"
    small_model: str = "anthropic/claude-haiku-4-5"

    # OpenRouter resilience – fallback if primary is unavailable
    openrouter_fallback_model: str = "anthropic/claude-sonnet-4-5"
```

```
# OpenRouter extra headers for routing and observability
OPENROUTER_EXTRA_HEADERS = {
    "HTTP-Referer": "https://emergeplatform.io",
    "X-Title": "MetaOrcha SuperAgent",
}
```

LangGraph configuration:

```
LANGGRAPH_CONFIG = {
    "recursion_limit": 50,      # Max 25 agent call round-trips per session
    "checkpoint_ttl": 86400,    # Session state expires after 24h inactivity
}
```

24. End-to-End Flow Walkthrough

Full Example: "Monitor ETH price and alert me if it drops below \$2800"

Turn 1 – User: "Monitor ETH price and alert me if it drops below \$2800"

LangGraph: receive_message → orchestrator_llm_node

PnD lookup: query="monitor ETH price alert threshold"

Returns: [price_oracle_mcp::get_price, system::set_price_alert, ...]

Orchestrator LLM thinks:

"The user wants a price alert, not real-time monitoring in this session.
I should clarify whether they want me to check now, or set up a recurring notification. Let me ask."

Response: "Just to confirm – would you like me to:

(A) Check the current ETH price right now, or

(B) Set up a recurring alert that notifies you whenever ETH drops below \$2800?"

[No tool calls – pure conversational response]

Turn 2 – User: "Option B, check every hour"

Orchestrator LLM thinks:

"The user wants a scheduled alert. I need to:

1. Get the current ETH price (to establish baseline)

2. Save this as a workflow template with hourly schedule

Let me create a checklist and start."

```
[tool_call: system::create_checklist({
  goal: "Monitor ETH price hourly, alert if below $2800",
  steps: [
    "Fetch current ETH price to establish baseline",
    "Save as recurring hourly workflow with $2800 alert threshold",
    "Confirm setup with user"
  ]
})]
[tool_call: price_oracle_mcp__get_price({asset: "ETH", chain: "BASE"})]
```

→ Execution Middleware:

InputGuard: asset="ETH" ✓, chain="BASE" ✓

PreFlight: price_oracle healthy ✓, API key in vault ✓

AuthResolver: injects API key

MCPHandler: calls get_price → returns {price: 2847.50, timestamp: ...}

OutputNormalizer: "Current ETH price: \$2,847.50"

```
[tool_call: system::update_checklist_step({step_id: 1, status: "done",
  result_summary: "Current ETH price: $2,847.50"})]
```

```
[tool_call: system::save_as_workflow_template({
  name: "ETH Price Alert",
  schedule_cron: "0 * * * *" // every hour
})]
```

LLM response: "Done! I've set up hourly ETH price monitoring. Current

```
price
            is $2,847.50. You'll be notified when it drops below
$2,800.
            Template saved as 'ETH Price Alert' (runs hourly)."
```

[One hour later – CronJob fires]

```
WorkflowScheduler creates new session, injects:
  "Execute workflow: Monitor ETH price hourly, alert if below $2800.
  Parameters: threshold=2800, asset=ETH"
```

```
Orchestrator LLM:
[tool_call: price_oracle_mcp__get_price({asset: "ETH"})]
→ receives: {price: 2785.20}

"ETH is at $2,785.20, which is below the $2,800 threshold."

[tool_call: system::send_notification({
  user_id: "...",
  message: "⚠ ETH Alert: Price dropped to $2,785.20 (below your $2,800
threshold)"
})]
```

```
Session completes. User receives push notification.
```

Appendix A: System Tool Registry (Complete)

Tool Name	Handler File	Output
system::create_checklist	checklist.py	Updated task_checklist in state
system::update_checklist_step	checklist.py	Updated step status + snapshot to history
system::add_checklist_step	checklist.py	Expanded checklist
system::update_checklist_scope	checklist.py	Revised checklist + snapshot
system::abandon_checklist	checklist.py	Cleared checklist
system::read_artifact	artifacts.py	Extracted text/data from binary artifact
system::save_as_workflow_template	workflow.py	Template ID persisted to Postgres
system::web_search	utils.py	Search results text (via system MCP)
system::get_current_datetime	utils.py	ISO 8601 timestamp

Tool Name	Handler File	Output
<code>system::send_notification</code>	<code>utils.py</code>	Push notification confirmation
<code>system::store_to_memory</code>	<code>memory.py</code>	Saved to Graphiti knowledge graph
<code>system::query_memory</code>	<code>memory.py</code>	Retrieved facts from Graphiti

Appendix B: Pre-Flight Check Summary

Check	When	Cached	On Failure
Manifest fetch	Every external call	5-min TTL (ManifestCache)	ValueError → error returned to LLM
Agent health ping	Every external call	30-sec TTL (Redis)	AgentUnreachableError → error to LLM with alternative suggestion
API key / Bearer	Every call (X_API_KEY, HTTP_BEARER)	No (vault lookup per call)	AuthRequiredError → AUTH_FORM_SUBMISSION interrupt
OAuth access token	Every OAuth2 call	Redis (token expiry - 5min)	Auto-refresh if refresh token exists; AUTH_CALLBACK if no refresh
OAuth2 DCR	First call per agent	Vault (permanent)	Auto-register, then follow OAuth2 path
Input schema	MCP TOOL calls only	Schema cached in manifest	InputValidationError → error returned to LLM (self-correct)
Destructiveness flag	Every external call	Manifest cache	ApprovalRequiredError → HITL_APPROVAL interrupt